

CHƯƠNG 17. AUTOENCODER, MẠNG ĐỐI KHÁNG SINH (GAN) VÀ MÔ HÌNH KHUẾCH TÁN

Autoencoder là các mạng thần kinh nhân tạo có khả năng học các biểu diễn dày đặc của dữ liệu đầu vào, được gọi là biểu diễn tiềm ẩn (latent representations) hoặc mã hóa (codings), mà không cần bất kỳ sự giám sát nào (tức là tập dữ liệu huấn luyện không được gán nhãn). Các mã hóa này thường có chiều thấp hơn nhiều so với dữ liệu đầu vào, làm cho autoencoder hữu ích cho việc giảm chiều (xem Chương 8), đặc biệt cho mục đích trực quan hóa. Autoencoder cũng hoạt động như bộ phát hiện đặc trưng, và chúng có thể được sử dụng để tiền huấn luyện không giám sát các mạng thần kinh sâu (như chúng ta đã thảo luận trong Chương 11). Cuối cùng, một số autoencoder là mô hình sinh (generative models): chúng có khả năng tạo ngẫu nhiên dữ liệu mới trông rất giống với dữ liệu huấn luyện. Ví dụ, bạn có thể huấn luyện một autoencoder trên ảnh khuôn mặt, và sau đó nó có thể tạo ra các khuôn mặt mới.

Mạng đối kháng sinh (GAN) cũng là mạng thần kinh có khả năng tạo dữ liệu. Trên thực tế, chúng có thể tạo ra những bức ảnh khuôn mặt thuyết phục đến mức khó tin rằng những người mà chúng đại diện không hề tồn tại. Bạn có thể tự mình đánh giá điều này bằng cách truy cập <https://thispersondoesnotexist.com>, một trang web hiển thị các khuôn mặt được tạo bởi kiến trúc GAN có tên là StyleGAN. Bạn cũng có thể xem <https://thisrentaldoesnotexist.com> để xem một số danh sách Airbnb được tạo. GAN hiện đang được sử dụng rộng rãi cho siêu phân giải (tăng độ phân giải của hình ảnh), tô màu, chỉnh sửa hình ảnh mạnh mẽ (ví dụ: thay thế những người làm hỏng ảnh bằng nền thực tế), biến các bản phác thảo đơn giản thành hình ảnh chân thực, dự đoán các khung hình tiếp theo trong video, tăng cường tập dữ liệu (để huấn luyện các mô hình khác), tạo các loại dữ liệu khác (như văn bản, âm thanh và chuỗi thời gian), xác định các điểm yếu trong các mô hình khác để tăng cường chúng, và nhiều hơn nữa.

Một bổ sung gần đây hơn vào nhóm học sinh là các mô hình khuếch tán (diffusion models). Năm 2021, chúng đã tạo ra những hình ảnh đa dạng và chất lượng cao hơn GAN, đồng thời cũng dễ huấn luyện hơn nhiều. Tuy nhiên, các mô hình khuếch tán chạy chậm hơn nhiều.

Autoencoder, GAN và mô hình khuếch tán đều không giám sát, chúng đều học các biểu diễn tiềm ẩn, chúng đều có thể được sử dụng làm mô hình sinh, và chúng có nhiều ứng dụng tương tự. Tuy nhiên, chúng hoạt động rất khác nhau:

Autoencoder đơn giản học cách sao chép đầu vào của chúng sang đầu ra. Điều này nghe có vẻ là một nhiệm vụ tầm thường, nhưng như bạn sẽ thấy, việc giới hạn mạng theo nhiều cách khác nhau có thể làm cho nó khá khó khăn. Ví dụ, bạn có thể giới hạn kích thước của các biểu diễn tiềm ẩn, hoặc bạn có thể thêm nhiễu vào đầu vào và huấn luyện mạng để khôi phục đầu vào gốc. Những ràng buộc này ngăn autoencoder sao chép trực tiếp đầu vào sang đầu ra một cách tầm thường, điều này buộc nó phải học các cách hiệu quả để biểu diễn dữ liệu. Tóm lại, các mã hóa là sản phẩm phụ của autoencoder học hàm đồng nhất dưới một số ràng buộc.

GAN bao gồm hai mạng thần kinh: một bộ tạo (generator) cố gắng tạo ra dữ liệu trông giống với dữ liệu huấn luyện, và một bộ phân biệt (discriminator) cố gắng phân biệt dữ liệu thật với dữ liệu giả. Kiến trúc này rất độc đáo trong học sâu ở chỗ bộ tạo và bộ phân biệt cạnh tranh với nhau trong quá trình huấn luyện: bộ tạo thường được so sánh với một tội phạm cố gắng tạo ra tiền giả giống thật, trong khi bộ phân biệt giống như điều tra viên cảnh sát cố gắng phân biệt tiền thật với tiền giả. Huấn luyện đối kháng (huấn luyện các mạng thần kinh cạnh tranh) được coi là một trong những đổi mới quan trọng nhất của những năm 2010. Năm 2016, Yann LeCun thậm chí đã nói rằng đó là “ý tưởng thú vị nhất trong 10 năm qua trong học máy”.

Một mô hình xác suất khuếch tán khử nhiễu (Denoising Diffusion Probabilistic Model - DDPM) được huấn luyện để loại bỏ một chút nhiễu khỏi một hình ảnh. Nếu bạn sau đó lấy một hình ảnh hoàn toàn đầy nhiễu Gaussian và lặp đi lặp lại chạy mô hình khuếch tán trên hình ảnh đó, một hình ảnh chất lượng cao sẽ dần dần xuất hiện, tương tự như các hình ảnh huấn luyện (nhưng không giống hệt).

Trong chương này, chúng ta sẽ bắt đầu bằng cách khám phá sâu hơn cách autoencoder hoạt động và cách sử dụng chúng để giảm chiều, trích xuất đặc trưng, tiền huấn luyện không giám sát, hoặc làm mô hình sinh. Điều này sẽ tự nhiên dẫn chúng ta đến GAN. Chúng ta sẽ xây dựng một GAN đơn giản để tạo ra hình ảnh giả, nhưng chúng ta sẽ thấy rằng việc huấn luyện thường khá khó khăn. Chúng ta sẽ thảo luận về những khó khăn chính mà bạn sẽ gặp phải khi huấn luyện đối kháng, cũng như một số kỹ thuật chính để khắc phục những khó khăn này. Và cuối cùng, chúng ta sẽ xây dựng và huấn luyện một DDPM và sử dụng nó để tạo ra hình ảnh. Hãy bắt đầu với autoencoder!

Biểu diễn dữ liệu hiệu quả

Trong các dãy số sau đây, bạn thấy dãy nào dễ nhớ nhất?

40, 27, 25, 36, 81, 57, 10, 73, 19, 68

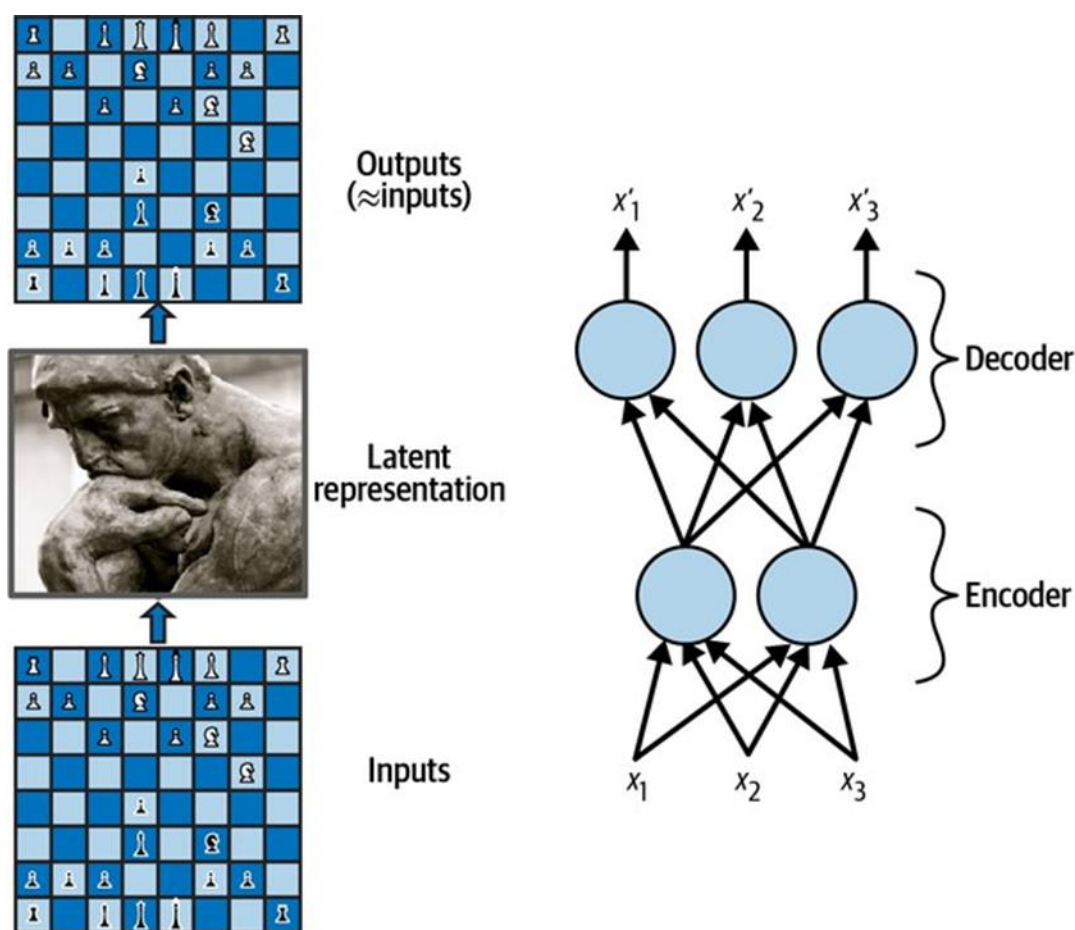
50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14

Thoạt nhìn, có vẻ như dãy đầu tiên sẽ dễ hơn, vì nó ngắn hơn nhiều. Tuy nhiên, nếu bạn nhìn kỹ dãy thứ hai, bạn sẽ nhận thấy rằng đó chỉ là danh sách các số chẵn từ 50 xuống 14. Một khi bạn nhận ra quy luật này, dãy thứ hai trở nên dễ nhớ hơn nhiều so với dãy đầu tiên vì bạn chỉ cần nhớ quy luật (tức là các số chẵn giảm dần) và các số bắt đầu và kết thúc (tức là 50 và 14). Lưu ý rằng nếu bạn có thể nhanh chóng và dễ dàng ghi nhớ các dãy rất dài, bạn sẽ không quan tâm nhiều đến sự tồn tại của một quy luật trong dãy thứ hai. Bạn sẽ chỉ học thuộc lòng từng số, và thế là xong. Thực tế là khó ghi nhớ các dãy dài khiến việc nhận ra các quy luật trở nên hữu ích, và hy vọng điều này làm rõ lý do tại sao việc ràng buộc một autoencoder trong quá trình huấn luyện thúc đẩy nó khám phá và khai thác các quy luật trong dữ liệu.

Mối quan hệ giữa trí nhớ, nhận thức và khớp mẫu đã được William Chase và Herbert Simon nổi tiếng nghiên cứu vào đầu những năm 1970. Họ quan sát thấy rằng các kỳ thủ cờ vua chuyên nghiệp có thể ghi nhớ vị trí của tất cả các quân cờ trong một ván đấu chỉ bằng cách nhìn vào bàn cờ trong năm giây, một nhiệm vụ mà hầu hết mọi người sẽ thấy không thể. Tuy nhiên, điều này chỉ đúng khi các quân cờ được đặt ở vị trí thực tế (từ các ván đấu thực tế),

chứ không phải khi các quân cờ được đặt ngẫu nhiên. Các chuyên gia cờ vua không có trí nhớ tốt hơn nhiều so với bạn và tôi; họ chỉ nhìn thấy các mẫu cờ vua dễ dàng hơn, nhờ kinh nghiệm của họ với trò chơi. Việc nhận thấy các mẫu giúp họ lưu trữ thông tin hiệu quả.

Giống như các kỳ thủ cờ vua trong thí nghiệm trí nhớ này, một autoencoder nhìn vào đầu vào, chuyển đổi chúng thành một biểu diễn tiềm ẩn hiệu quả, và sau đó cho ra thứ gì đó (hy vọng) trông rất giống với đầu vào. Một autoencoder luôn bao gồm hai phần: một bộ mã hóa (encoder) (hoặc mạng nhận dạng) chuyển đổi đầu vào thành một biểu diễn tiềm ẩn, tiếp theo là một bộ giải mã (decoder) (hoặc mạng sinh) chuyển đổi biểu diễn nội bộ thành đầu ra (xem Hình 17-1).



Hình 17-1. Thí nghiệm trí nhớ cờ vua (trái) và một autoencoder đơn giản (phải)

Như bạn có thể thấy, một autoencoder thường có kiến trúc giống như một perceptron đa lớp (MLP; xem Chương 10), ngoại trừ số lượng neuron trong lớp đầu ra phải bằng số lượng đầu vào. Trong ví dụ này, chỉ có một lớp ẩn bao gồm hai neuron (bộ mã hóa), và một lớp đầu ra bao gồm ba neuron (bộ giải mã). Các đầu ra thường được gọi là các bản tái tạo (reconstructions) vì autoencoder cố gắng tái tạo lại đầu vào. Hàm chi phí chứa một mất tái tạo (reconstruction loss) để phạt mô hình khi các bản tái tạo khác với đầu vào.

Vì biểu diễn nội bộ có chiều thấp hơn dữ liệu đầu vào (nó là 2D thay vì 3D), autoencoder được gọi là thiếu hoàn chỉnh (undercomplete). Một autoencoder thiếu hoàn chỉnh không thể sao chép trực tiếp đầu vào của nó sang các mã hóa, nhưng nó phải tìm cách xuất ra một bản sao của đầu vào của nó. Nó buộc phải học các đặc trưng quan trọng nhất trong dữ liệu đầu vào (và bỏ qua những đặc trưng không quan trọng).

Hãy xem cách triển khai một autoencoder thiếu hoàn chỉnh rất đơn giản để giảm chiều.

Thực hiện PCA bằng Autoencoder tuyến tính thiếu hoàn chỉnh

Nếu autoencoder chỉ sử dụng các kích hoạt tuyến tính và hàm chi phí là sai số bình phương trung bình (MSE), thì nó sẽ thực hiện phân tích thành phần chính (PCA; xem Chương 8).

Đoạn mã sau đây xây dựng một autoencoder tuyến tính đơn giản để thực hiện PCA trên tập dữ liệu 3D, chiếu nó thành 2D:

```
import tensorflow as tf

encoder = tf.keras.Sequential([tf.keras.layers.Dense(2)])
decoder = tf.keras.Sequential([tf.keras.layers.Dense(3)])
autoencoder = tf.keras.Sequential([encoder, decoder])

optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)
autoencoder.compile(loss="mse", optimizer=optimizer)
```

Mã này thực sự không khác nhiều so với tất cả các MLP mà chúng ta đã xây dựng trong các chương trước, nhưng có một vài điều cần lưu ý:

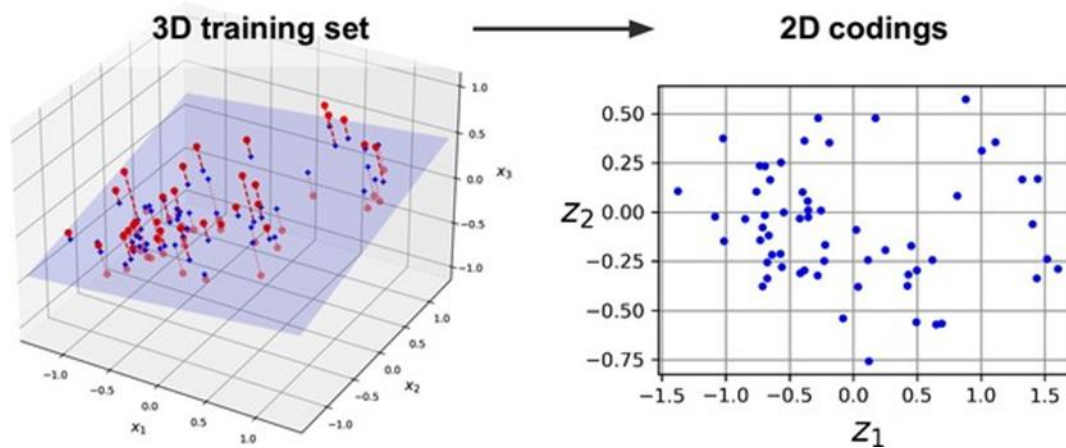
- Chúng ta đã tổ chức autoencoder thành hai thành phần con: bộ mã hóa (encoder) và bộ giải mã (decoder). Cả hai đều là các mô hình Sequential thông thường với một lớp Dense duy nhất, và autoencoder là một mô hình Sequential chứa bộ mã hóa theo sau là bộ giải mã (hãy nhớ rằng một mô hình có thể được sử dụng làm một lớp trong một mô hình khác).
- Số lượng đầu ra của autoencoder bằng số lượng đầu vào (tức là 3).
- Để thực hiện PCA, chúng ta không sử dụng bất kỳ hàm kích hoạt nào (tức là tất cả các neuron đều tuyến tính), và hàm chi phí là MSE. Điều này là do PCA là một phép biến đổi tuyến tính. Chúng ta sẽ sớm thấy các autoencoder phức tạp hơn và phi tuyến tính.

Bây giờ, hãy huấn luyện mô hình trên cùng một tập dữ liệu 3D đơn giản được tạo mà chúng ta đã sử dụng trong Chương 8 và sử dụng nó để mã hóa tập dữ liệu đó (tức là chiếu nó thành 2D):

```
X_train = [...] #generate a 3D dataset, like in Chapter 8

history = autoencoder.fit(X_train, X_train, epochs=500, verbose=False)
codings = encoder.predict(X_train)
```

Lưu ý rằng X_{train} được sử dụng làm cả đầu vào và mục tiêu. Hình 17-2 cho thấy tập dữ liệu 3D gốc (bên trái) và đầu ra của lớp ẩn của autoencoder (tức là lớp mã hóa, bên phải). Như bạn có thể thấy, autoencoder đã tìm thấy mặt phẳng 2D tốt nhất để chiếu dữ liệu lên đó, bảo toàn nhiều phương sai trong dữ liệu nhất có thể (giống như PCA).

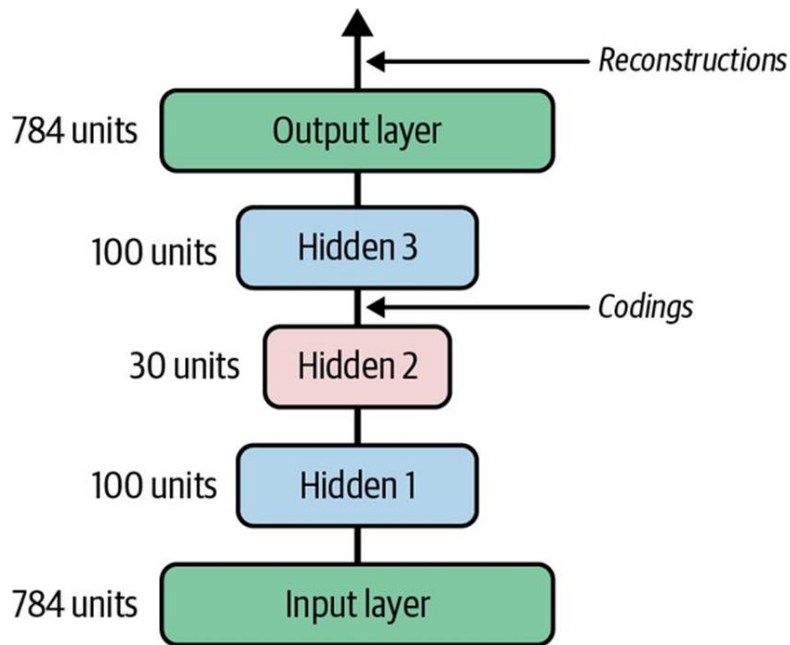


Hình 17-2. PCA gần đúng được thực hiện bởi một autoencoder tuyến tính thiếu hoàn chỉnh

Autoencoder xếp chồng (Stacked Autoencoders)

Giống như các mạng thần kinh khác mà chúng ta đã thảo luận, autoencoder có thể có nhiều lớp ẩn. Trong trường hợp này, chúng được gọi là autoencoder xếp chồng (stacked autoencoders) (hoặc autoencoder sâu). Việc thêm nhiều lớp giúp autoencoder học được các mã hóa phức tạp hơn. Điều đó nói lên rằng, người ta phải cẩn thận không làm cho autoencoder quá mạnh mẽ. Hãy tưởng tượng một bộ mã hóa mạnh đến mức nó chỉ học cách ánh xạ từng đầu vào tới một số tùy ý duy nhất (và bộ giải mã học ánh xạ ngược lại). Rõ ràng một autoencoder như vậy sẽ tái tạo dữ liệu huấn luyện một cách hoàn hảo, nhưng nó sẽ không học được bất kỳ biểu diễn dữ liệu hữu ích nào trong quá trình đó, và nó không có khả năng khái quát hóa tốt cho các trường hợp mới.

Kiến trúc của một autoencoder xếp chồng thường đối xứng so với lớp ẩn trung tâm (lớp mã hóa). Nói một cách đơn giản, nó trông giống như một chiếc bánh sandwich. Ví dụ, một autoencoder cho Fashion MNIST (được giới thiệu trong Chương 10) có thể có 784 đầu vào, tiếp theo là một lớp ẩn với 100 neuron, sau đó là một lớp ẩn trung tâm gồm 30 neuron, sau đó là một lớp ẩn khác với 100 neuron, và một lớp đầu ra với 784 neuron. Autoencoder xếp chồng này được biểu diễn trong Hình 17-3.



Hình 17-3. Autoencoder xếp chồng

Triển khai Autoencoder xếp chồng bằng Keras

Bạn có thể triển khai một autoencoder xếp chồng rất giống với một MLP sâu thông thường:

```

stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
stacked_ae = tf.keras.Sequential([stacked_encoder, stacked_decoder])

stacked_ae.compile(loss="mse", optimizer="nadam")
history = stacked_ae.fit(X_train, X_train, epochs=20,
                        validation_data=(X_valid, X_valid))
  
```

Hãy xem qua đoạn mã này:

- Cũng như trước đây, chúng ta chia mô hình autoencoder thành hai mô hình con: bộ mã hóa (encoder) và bộ giải mã (decoder).
- Bộ mã hóa nhận các hình ảnh thang độ xám 28×28 pixel, làm phẳng chúng để mỗi hình ảnh được biểu diễn dưới dạng một vector có kích thước 784, sau đó xử lý các vector này thông qua hai lớp Dense với kích thước giảm dần (100 đơn vị sau đó 30 đơn

vi), cả hai đều sử dụng hàm kích hoạt ReLU. Đối với mỗi hình ảnh đầu vào, bộ mã hóa xuất ra một vector có kích thước 30.

- Bộ giải mã nhận các mã hóa có kích thước 30 (được xuất bởi bộ mã hóa) và xử lý chúng thông qua hai lớp Dense với kích thước tăng dần (100 đơn vị sau đó 784 đơn vị), và nó định hình lại các vector cuối cùng thành các mảng 28×28 để đầu ra của bộ giải mã có cùng hình dạng với đầu vào của bộ mã hóa.
- Khi biên dịch autoencoder xếp chồng, chúng ta sử dụng hàm mất MSE và tối ưu hóa Nadam.
- Cuối cùng, chúng ta huấn luyện mô hình bằng cách sử dụng `X_train` làm cả đầu vào và mục tiêu. Tương tự, chúng ta sử dụng `X_valid` làm cả đầu vào và mục tiêu xác thực.

Để dịch phần còn lại của nội dung sang tiếng Việt, tôi sẽ tiếp tục dịch từng phần nhỏ để đảm bảo tính chính xác và mạch lạc. Tôi cũng sẽ chú ý đến các công thức toán học và cấu trúc mã nguồn để giữ nguyên định dạng.

Trực quan hóa các bản tái tạo

Một cách để đảm bảo rằng autoencoder được huấn luyện đúng cách là so sánh đầu vào và đầu ra: sự khác biệt không nên quá đáng kể. Hãy vẽ một vài hình ảnh từ tập xác thực, cũng như các bản tái tạo của chúng:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_reconstructions(model, images=X_valid, n_images=5):
    # Đảm bảo đầu ra nằm trong khoảng [0, 1] để hiển thị ảnh
    reconstructions = np.clip(model.predict(images[:n_images]), 0, 1)

    fig = plt.figure(figsize=(n_images * 1.5, 3))
    for image_index in range(n_images):
        plt.subplot(2, n_images, 1 + image_index)
        plt.imshow(images[image_index], cmap="binary")
        plt.axis("off")
        plt.subplot(2, n_images, 1 + n_images + image_index)
        plt.imshow(reconstructions[image_index], cmap="binary")
        plt.axis("off")

# Đoạn mã này cần một môi trường Python với TensorFlow, NumPy và Matplotlib
# cùng với các biến X_valid và stacked_ae được định nghĩa trước đó để chạy.
# plot_reconstructions(stacked_ae)
# plt.show()
```

Hình 17-4 cho thấy các hình ảnh kết quả.



Hình 17-4. Hình ảnh gốc (trên cùng) và các bản tái tạo của chúng (dưới cùng)

Các bản tái tạo có thể nhận ra được, nhưng hơi mất mát quá nhiều. Chúng ta có thể cần huấn luyện mô hình lâu hơn, hoặc làm cho bộ mã hóa và bộ giải mã sâu hơn, hoặc làm cho các mã hóa lớn hơn. Nhưng nếu chúng ta làm cho mạng quá mạnh, nó sẽ quản lý để tạo ra các bản tái tạo hoàn hảo mà không học được bất kỳ mẫu hữu ích nào trong dữ liệu. Hiện tại, hãy tiếp tục với mô hình này.

Trực quan hóa tập dữ liệu Fashion MNIST

Bây giờ chúng ta đã huấn luyện một autoencoder xếp chồng, chúng ta có thể sử dụng nó để giảm chiều của tập dữ liệu. Đối với việc trực quan hóa, điều này không mang lại kết quả tuyệt vời so với các thuật toán giảm chiều khác (chẳng hạn như những gì chúng ta đã thảo luận trong Chương 8), nhưng một lợi thế lớn của autoencoder là chúng có thể xử lý các tập dữ liệu lớn với nhiều trường hợp và nhiều đặc trưng. Vì vậy, một chiến lược là sử dụng autoencoder để giảm chiều xuống một mức hợp lý, sau đó sử dụng một thuật toán giảm chiều khác để trực quan hóa. Hãy sử dụng chiến lược này để trực quan hóa Fashion MNIST. Đầu tiên chúng ta sẽ sử dụng bộ mã hóa từ autoencoder xếp chồng của chúng ta để giảm chiều xuống 30, sau đó chúng ta sẽ sử dụng triển khai thuật toán t-SNE của Scikit-Learn để giảm chiều xuống 2 để trực quan hóa:

```
from sklearn.manifold import TSNE
# import numpy as np # Đảm bảo import numpy nếu chưa có
# import matplotlib.pyplot as plt # Đảm bảo import matplotlib.pyplot nếu chưa có

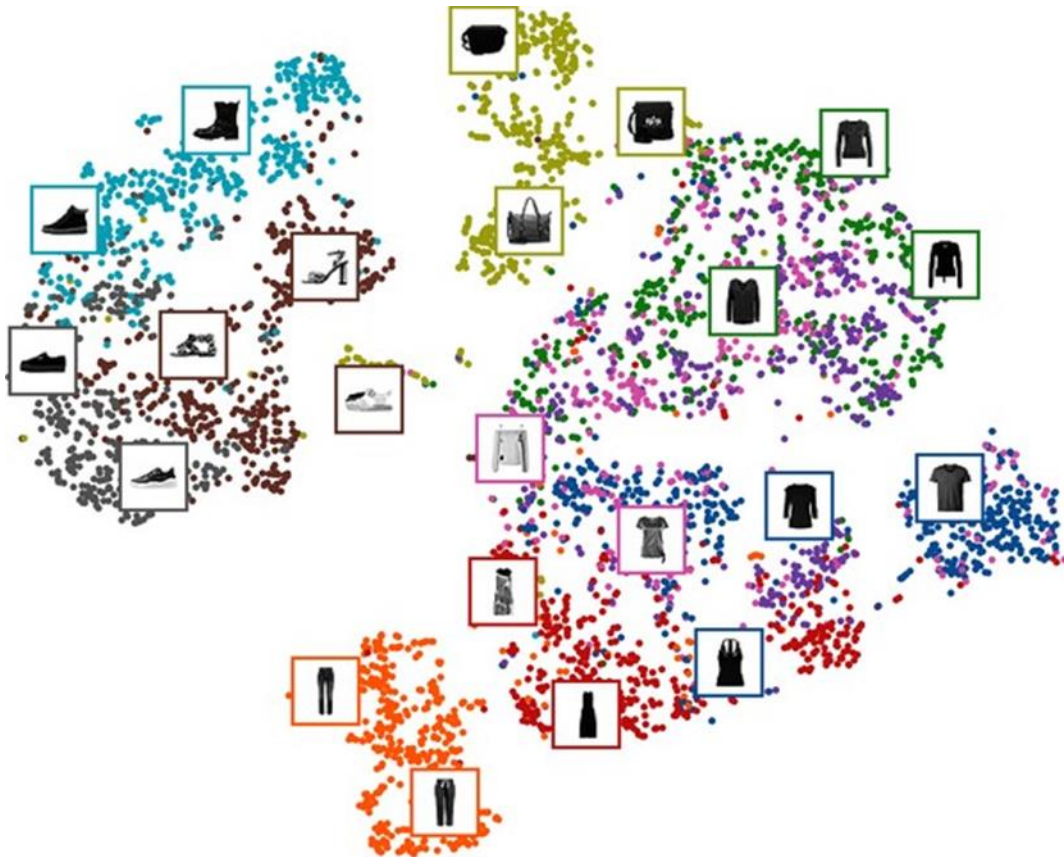
# Giả định X_valid và y_valid đã được định nghĩa và có sẵn từ ngữ cảnh trước đó
# Giả định stacked_encoder đã được huấn luyện và có sẵn từ ngữ cảnh trước đó

# X_valid_compressed = stacked_encoder.predict(X_valid)
# tsne = TSNE(init="pca", learning_rate="auto", random_state=42)
# X_valid_2D = tsne.fit_transform(X_valid_compressed)
```

Bây giờ chúng ta có thể vẽ biểu đồ tập dữ liệu:

```
# plt.scatter(X_valid_2D[:, 0], X_valid_2D[:, 1], c=y_valid, s=10, cmap="tab10")
# plt.show()
```

Hình 17-5 cho thấy biểu đồ phân tán kết quả, được làm đẹp một chút bằng cách hiển thị một số hình ảnh. Thuật toán t-SNE đã xác định một số cụm khớp với các lớp khá tốt (mỗi lớp được biểu thị bằng một màu khác nhau).



Hình 17-5. Trực quan hóa Fashion MNIST bằng cách sử dụng autoencoder sau đó là t-SNE

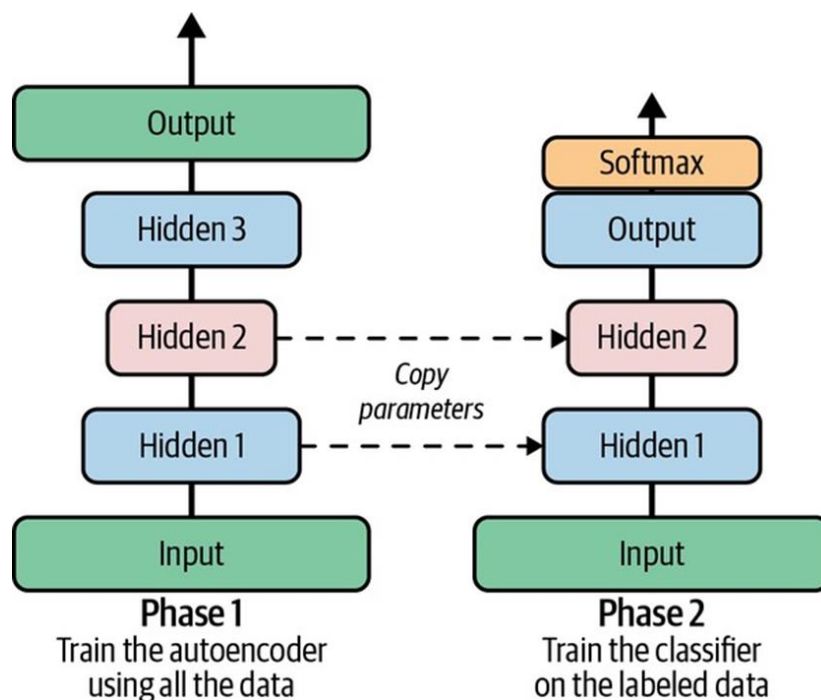
Vì vậy, autoencoder có thể được sử dụng để giảm chiều. Một ứng dụng khác là để tiền huấn luyện không giám sát.

Tiền huấn luyện không giám sát bằng Autoencoder xếp chồng

Như chúng ta đã thảo luận trong Chương 11, nếu bạn đang giải quyết một tác vụ giám sát phức tạp nhưng bạn không có nhiều dữ liệu huấn luyện được gán nhãn, một giải pháp là tìm một mạng thần kinh thực hiện một tác vụ tương tự và sử dụng lại các lớp thấp hơn của nó. Điều này giúp có thể huấn luyện một mô hình hiệu suất cao bằng cách sử dụng ít dữ liệu huấn luyện vì mạng thần kinh của bạn sẽ không phải học tất cả các đặc trưng cấp thấp; nó sẽ chỉ sử dụng lại các bộ phát hiện đặc trưng được học bởi mạng hiện có.

Tương tự, nếu bạn có một tập dữ liệu lớn nhưng hầu hết không được gán nhãn, bạn có thể huấn luyện trước một autoencoder xếp chồng bằng cách sử dụng tất cả dữ liệu, sau đó sử

dùng lại các lớp thấp hơn để tạo một mạng thần kinh cho tác vụ thực tế của bạn và huấn luyện nó bằng cách sử dụng dữ liệu được gán nhãn. Ví dụ, Hình 17-6 cho thấy cách sử dụng một autoencoder xếp chồng để thực hiện tiền huấn luyện không giám sát cho một mạng thần kinh phân loại. Khi huấn luyện bộ phân loại, nếu bạn thực sự không có nhiều dữ liệu huấn luyện được gán nhãn, bạn có thể muốn đóng băng các lớp được tiền huấn luyện (ít nhất là các lớp thấp hơn).



Hình 17-6. Tiền huấn luyện không giám sát bằng cách sử dụng autoencoder

Không có gì đặc biệt về việc triển khai: chỉ cần huấn luyện một autoencoder bằng cách sử dụng tất cả dữ liệu huấn luyện (có gán nhãn và không gán nhãn), sau đó sử dụng lại các lớp mã hóa của nó để tạo một mạng thần kinh mới (xem các bài tập ở cuối chương này để biết ví dụ).

Tiếp theo, hãy xem xét một vài kỹ thuật để huấn luyện autoencoder xếp chồng.

Ràng buộc trọng số (Tying Weights)

Khi một autoencoder đối xứng gọn gàng, giống như cái chúng ta vừa xây dựng, một kỹ thuật phổ biến là ràng buộc trọng số của các lớp giải mã với trọng số của các lớp mã hóa. Điều này làm giảm một nửa số trọng số trong mô hình, tăng tốc độ huấn luyện và hạn chế rủi ro quá khớp. Cụ thể, nếu autoencoder có tổng số N lớp (không tính lớp đầu vào), và w_L biểu thị trọng số kết nối của lớp thứ L (ví dụ: lớp 1 là lớp ẩn đầu tiên, lớp $N/2$ là lớp mã hóa, và lớp N là lớp đầu ra), thì trọng số lớp giải mã có thể được định nghĩa là $w_L = w_{N-L+1}^T$ (với $L = N/2 + 1, \dots, N$).

Để ràng buộc trọng số giữa các lớp bằng Keras, hãy định nghĩa một lớp tùy chỉnh:

```
import tensorflow as tf

class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.biases = self.add_weight(name="bias",
                                       shape=self.dense.input_shape[-1],
                                       initializer="zeros")
        super().build(batch_input_shape)

    def call(self, inputs):
        Z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(Z + self.biases)
```

Lớp tùy chỉnh này hoạt động giống như một lớp Dense thông thường, nhưng nó sử dụng trọng số của một lớp Dense khác, được chuyển vị (đặt `transpose_b=True` tương đương với việc chuyển vị đối số thứ hai, nhưng hiệu quả hơn vì nó thực hiện phép chuyển vị ngay lập tức trong hoạt động `matmul()`). Tuy nhiên, nó sử dụng vector độ lệch riêng của nó. Bây giờ chúng ta có thể xây dựng một autoencoder xếp chồng mới, rất giống với cái trước nhưng với các lớp Dense của bộ giải mã được ràng buộc với các lớp Dense của bộ mã hóa:

```
import tensorflow as tf

dense_1 = tf.keras.layers.Dense(100, activation="relu")
dense_2 = tf.keras.layers.Dense(30, activation="relu")

tied_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    dense_1,
    dense_2
])

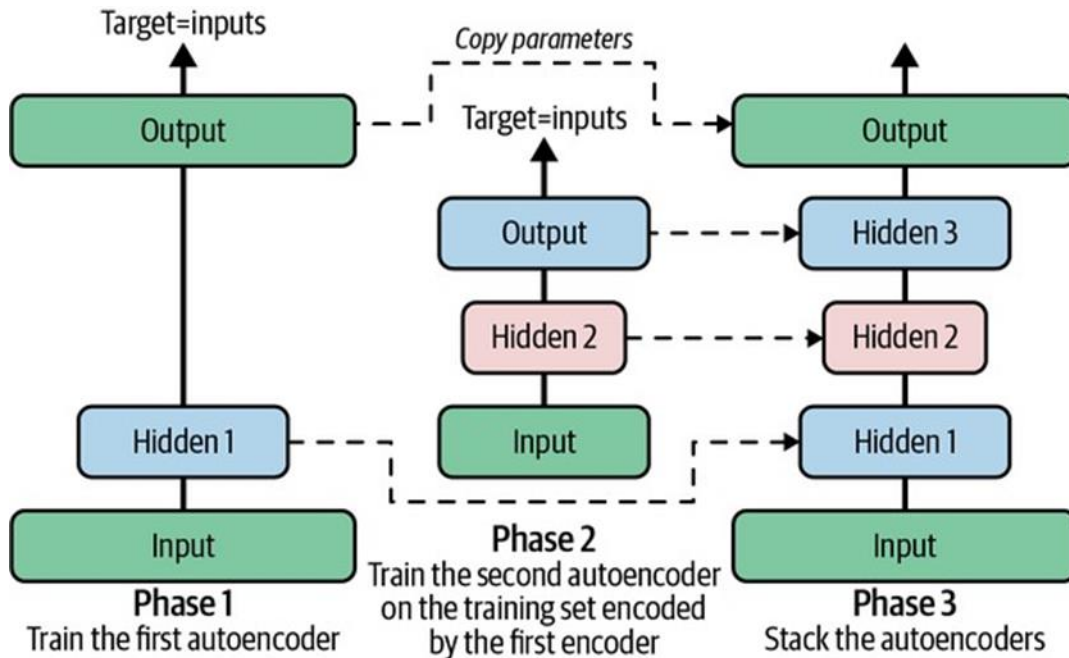
tied_decoder = tf.keras.Sequential([
    DenseTranspose(dense_2, activation="relu"),
    DenseTranspose(dense_1),
    tf.keras.layers.Reshape([28, 28])
])

tied_ae = tf.keras.Sequential([tied_encoder, tied_decoder])
```

Mô hình này đạt được sai số tái tạo xấp xỉ tương tự như mô hình trước, sử dụng gần một nửa số tham số.

Huấn luyện từng Autoencoder một

Thay vì huấn luyện toàn bộ autoencoder xếp chồng trong một lần như chúng ta vừa làm, có thể huấn luyện từng autoencoder nông một, sau đó xếp chồng tất cả chúng thành một autoencoder xếp chồng duy nhất (do đó có tên gọi này), như được hiển thị trong Hình 17-7. Kỹ thuật này ngày nay không được sử dụng nhiều, nhưng bạn vẫn có thể gặp các bài báo nói về “greedy layerwise training”, vì vậy điều quan trọng là phải biết ý nghĩa của nó.



Hình 17-7. Huấn luyện từng autoencoder một

Trong giai đoạn huấn luyện đầu tiên, autoencoder đầu tiên học cách tái tạo đầu vào. Sau đó, chúng ta mã hóa toàn bộ tập huấn luyện bằng cách sử dụng autoencoder đầu tiên này, và điều này mang lại cho chúng ta một tập huấn luyện mới (đã nén). Chúng ta sau đó huấn luyện autoencoder thứ hai trên tập dữ liệu mới này. Đây là giai đoạn huấn luyện thứ hai. Cuối cùng, chúng ta xây dựng một “chiếc bánh sandwich” lớn bằng cách sử dụng tất cả các autoencoder này, như được hiển thị trong Hình 17-7 (tức là chúng ta đầu tiên xếp chồng các lớp ẩn của mỗi autoencoder, sau đó là các lớp đầu ra theo thứ tự ngược lại). Điều này mang lại cho chúng ta autoencoder xếp chồng cuối cùng (xem phần “Huấn luyện từng Autoencoder một” trong sổ tay của chương để biết ví dụ triển khai). Chúng ta có thể dễ dàng huấn luyện nhiều autoencoder hơn theo cách này, xây dựng một autoencoder xếp chồng rất sâu.

Như tôi đã đề cập trước đây, một trong những yếu tố kích hoạt “sóng thần học sâu” là việc phát hiện vào năm 2006 bởi Geoffrey Hinton và cộng sự rằng các mạng thần kinh sâu có thể được tiền huấn luyện một cách không giám sát, sử dụng phương pháp “greedy layerwise” này. Họ đã sử dụng máy Boltzmann hạn chế (RBM; xem <https://homl.info/extra-anns>) cho mục đích này, nhưng vào năm 2007 Yoshua Bengio và cộng sự đã chỉ ra rằng autoencoder cũng hoạt động tốt. Trong vài năm, đây là cách duy nhất hiệu quả để huấn luyện các mạng

sâu, cho đến khi nhiều kỹ thuật được giới thiệu trong Chương 11 giúp có thể huấn luyện một mạng sâu trong một lần.

Autoencoder không giới hạn ở các mạng dày đặc: bạn cũng có thể xây dựng autoencoder tích chập. Hãy xem xét chúng bây giờ.

Autoencoder tích chập (Convolutional Autoencoders)

Nếu bạn đang xử lý hình ảnh, thì các autoencoder chúng ta đã thấy cho đến nay sẽ không hoạt động tốt (trừ khi hình ảnh rất nhỏ): như bạn đã thấy trong Chương 14, mạng thần kinh tích chập phù hợp hơn nhiều so với mạng dày đặc để làm việc với hình ảnh. Vì vậy, nếu bạn muốn xây dựng một autoencoder cho hình ảnh (ví dụ: để tiền huấn luyện không giám sát hoặc giảm chiều), bạn sẽ cần xây dựng một autoencoder tích chập. Bộ mã hóa là một CNN thông thường bao gồm các lớp tích chập và lớp gộp. Nó thường làm giảm chiều không gian của đầu vào (tức là chiều cao và chiều rộng) trong khi tăng chiều sâu (tức là số lượng bản đồ đặc trưng). Bộ giải mã phải làm ngược lại (phóng to hình ảnh và giảm chiều sâu của nó trở lại kích thước gốc), và đối với điều này, bạn có thể sử dụng các lớp tích chập chuyển vị (transpose convolutional layers) (cách khác, bạn có thể kết hợp các lớp lấy mẫu lên (upsampling layers) với các lớp tích chập). Dưới đây là một autoencoder tích chập cơ bản cho Fashion MNIST:

```
import tensorflow as tf

conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 14 x 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 7 x 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 3 x 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAvgPool2D() # output: 30
])

conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
                                     activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"),
    tf.keras.layers.Reshape([28, 28])
])

conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])
```

Cũng có thể tạo autoencoder với các loại kiến trúc khác, chẳng hạn như RNN (xem sổ tay để biết ví dụ).

Được rồi, hãy lùi lại một chút. Cho đến nay, chúng ta đã xem xét các loại autoencoder khác nhau (cơ bản, xếp chồng và tích chập), và cách huấn luyện chúng (hoặc trong một lần hoặc từng lớp một). Chúng ta cũng đã xem xét một vài ứng dụng: trực quan hóa dữ liệu và tiền huấn luyện không giám sát.

Cho đến bây giờ, để buộc autoencoder học các đặc trưng thú vị, chúng ta đã giới hạn kích thước của lớp mã hóa, làm cho nó thiếu hoàn chỉnh. Thực tế có nhiều loại ràng buộc khác có thể được sử dụng, bao gồm những ràng buộc cho phép lớp mã hóa lớn bằng đầu vào, hoặc thậm chí lớn hơn, dẫn đến một autoencoder thừa hoàn chỉnh (overcomplete). Sau đó, trong các phần sau, chúng ta sẽ xem xét thêm một vài loại autoencoder: autoencoder khử nhiễu (denoising autoencoders), autoencoder thưa (sparse autoencoders) và autoencoder biến phân (variational autoencoders).

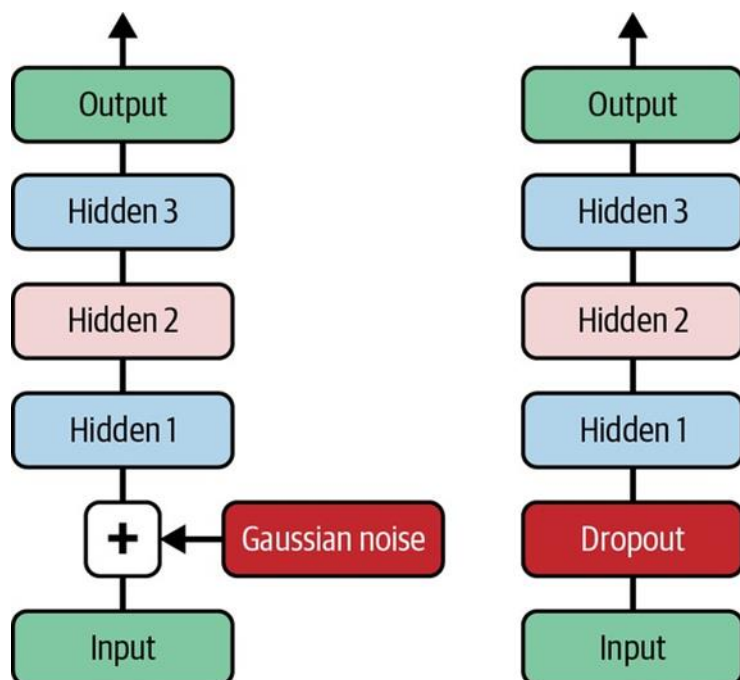
Autoencoder khử nhiễu (Denoising Autoencoders)

Một cách khác để buộc autoencoder học các đặc trưng hữu ích là thêm nhiễu vào đầu vào của nó, huấn luyện nó để khôi phục lại các đầu vào gốc, không nhiễu. Ý tưởng này đã tồn tại từ những năm 1980 (ví dụ, nó được đề cập trong luận văn thạc sĩ năm 1987 của Yann LeCun). Trong một bài báo năm 2008, Pascal Vincent và cộng sự đã chỉ ra rằng autoencoder cũng có thể được sử dụng để trích xuất đặc trưng. Trong một bài báo năm 2010, Vincent và cộng sự đã giới thiệu autoencoder khử nhiễu xếp chồng.

Nhiễu có thể là nhiễu Gaussian thuần túy được thêm vào đầu vào, hoặc nó có thể là các đầu vào bị tắt ngẫu nhiên, giống như trong dropout (được giới thiệu trong Chương 11). Hình 17-8 cho thấy cả hai tùy chọn. Việc triển khai rất đơn giản: đó là một autoencoder xếp chồng thông thường với một lớp Dropout bổ sung được áp dụng cho đầu vào của bộ mã hóa (hoặc bạn có thể sử dụng lớp GaussianNoise thay thế). <1>Hãy nhớ rằng lớp Dropout chỉ hoạt động trong quá trình huấn luyện (và lớp GaussianNoise cũng vậy):</1>

```
import tensorflow as tf

dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5), # Tỷ lệ dropout 0.5
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu")
])
dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
dropout_ae = tf.keras.Sequential([dropout_encoder, dropout_decoder])
```



Hình 17-8. Autoencoder khử nhiễu, với nhiễu Gaussian (trái) hoặc dropout (phải)

Hình 17-9 cho thấy một vài hình ảnh bị nhiễu (với một nửa số pixel bị tắt), và các hình ảnh được tái tạo bởi autoencoder khử nhiễu dựa trên dropout.

Lưu ý cách autoencoder đoán các chi tiết thực sự không có trong đầu vào, chẳng hạn như phần trên của chiếc áo sơ mi trắng (hàng dưới cùng, hình thứ tư). Như bạn có thể thấy, autoencoder khử nhiễu không chỉ có thể được sử dụng để trực quan hóa dữ liệu hoặc tiền huấn luyện không giám sát, giống như các autoencoder khác mà chúng ta đã thảo luận cho đến nay, mà chúng còn có thể được sử dụng khá đơn giản và hiệu quả để loại bỏ nhiễu khỏi hình ảnh.



Hình 17-9. Hình ảnh bị nhiễu (trên cùng) và các bản tái tạo của chúng (dưới cùng)

Autoencoder thưa (Sparse Autoencoders)

Một loại ràng buộc khác thường dẫn đến trích xuất đặc trưng tốt là tính thưa thớt: bằng cách thêm một thuật ngữ thích hợp vào hàm chi phí, autoencoder được đẩy để giảm số lượng neuron hoạt động trong lớp mã hóa. Ví dụ, nó có thể được đẩy để trung bình chỉ có 5% các neuron hoạt động đáng kể trong lớp mã hóa. Điều này buộc autoencoder phải biểu diễn mỗi đầu vào như một sự kết hợp của một số lượng nhỏ các kích hoạt. Kết quả là, mỗi neuron trong lớp mã hóa thường kết thúc biểu diễn một đặc trưng hữu ích (nếu bạn chỉ có thể nói vài từ mỗi tháng, bạn có lẽ sẽ cố gắng làm cho chúng đáng nghe).

Một cách tiếp cận đơn giản là sử dụng hàm kích hoạt sigmoid trong lớp mã hóa (để ràng buộc các mã hóa với các giá trị từ 0 đến 1), sử dụng một lớp mã hóa lớn (ví dụ, với 300 đơn vị), và thêm một số chuẩn hóa L1 vào các kích hoạt của lớp mã hóa. Bộ giải mã chỉ là một bộ giải mã thông thường:

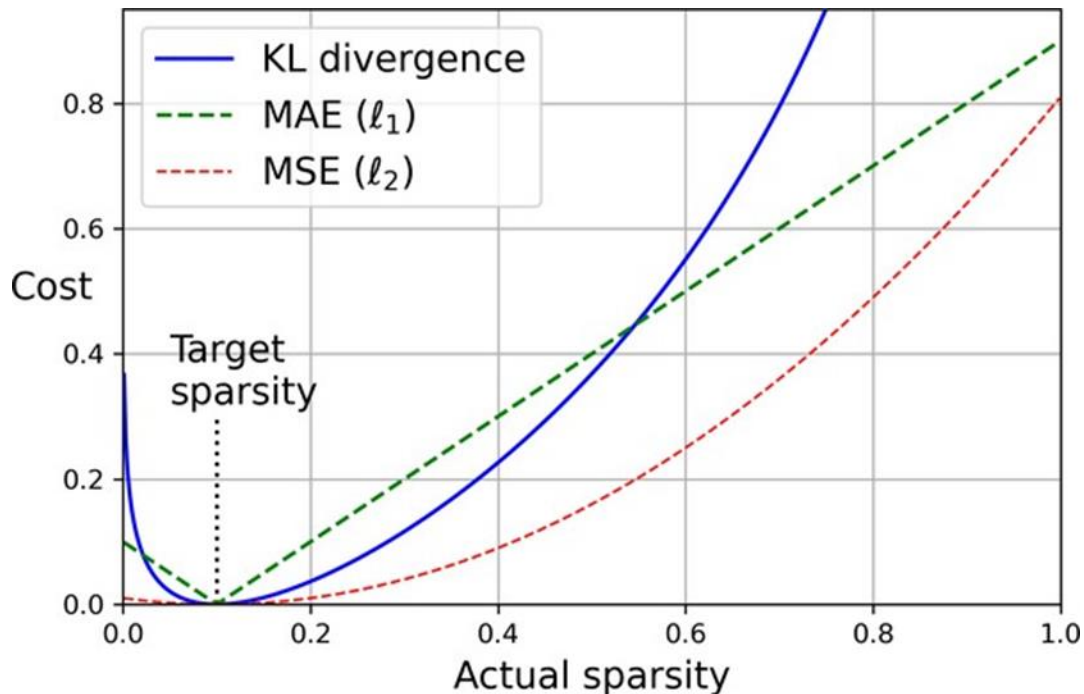
```
import tensorflow as tf

sparse_l1_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid"),
    tf.keras.layers.ActivityRegularization(l1=1e-4) # Chuẩn hóa L1
])
sparse_l1_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
sparse_l1_ae = tf.keras.Sequential([sparse_l1_encoder, sparse_l1_decoder])
```

Lớp ActivityRegularization này chỉ trả về đầu vào của nó, nhưng như một tác dụng phụ, nó thêm một mất huấn luyện bằng tổng các giá trị tuyệt đối của đầu vào của nó. Điều này chỉ ảnh hưởng đến quá trình huấn luyện. Tương đương, bạn có thể loại bỏ lớp ActivityRegularization và đặt activity_regularizer=tf.keras.regularizers.l1(1e-4) trong lớp trước đó. Hình phạt này sẽ khuyến khích mạng thần kinh tạo ra các mã hóa gần 0, nhưng vì nó cũng sẽ bị phạt nếu không tái tạo đúng đầu vào, nó sẽ phải xuất ít nhất một vài giá trị khác 0. Việc sử dụng chuẩn L1 thay vì chuẩn L2 sẽ đẩy mạng thần kinh bảo toàn các mã hóa quan trọng nhất trong khi loại bỏ những mã hóa không cần thiết cho hình ảnh đầu vào (thay vì chỉ giảm tất cả các mã hóa).

Một cách tiếp cận khác, thường mang lại kết quả tốt hơn, là đo lường tính thưa thớt thực tế của lớp mã hóa ở mỗi lần lặp huấn luyện và phạt mô hình khi tính thưa thớt đo được khác với tính thưa thớt mục tiêu. Chúng ta làm như vậy bằng cách tính kích hoạt trung bình của mỗi neuron trong lớp mã hóa, trên toàn bộ batch huấn luyện. Kích thước batch không được quá nhỏ, nếu không giá trị trung bình sẽ không chính xác.

Một khi chúng ta có kích hoạt trung bình trên mỗi neuron, chúng ta muốn phạt các neuron quá hoạt động, hoặc không đủ hoạt động, bằng cách thêm một mất thừa thớt vào hàm chi phí. Ví dụ, nếu chúng ta đo được rằng một neuron có kích hoạt trung bình là 0.3, nhưng tính thừa thớt mục tiêu là 0.1, nó phải bị phạt để kích hoạt ít hơn. Một cách tiếp cận có thể là đơn giản thêm sai số bình phương $(0.3 - 0.1)^2$ vào hàm chi phí, nhưng trong thực tế, một cách tiếp cận tốt hơn là sử dụng độ phân kỳ Kullback–Leibler (KL) (đã được thảo luận ngắn gọn trong Chương 4), có gradient mạnh hơn nhiều so với sai số bình phương trung bình, như bạn có thể thấy trong Hình 17-10.



Hình 17-10. Mất thừa thớt

Với hai phân phối xác suất rời rạc P và Q , độ phân kỳ KL giữa các phân phối này, ký hiệu $D_{\text{KL}}(P \parallel Q)$, có thể được tính bằng Công thức 17-1.

Công thức 17-1. Độ phân kỳ Kullback–Leibler

$$D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Trong trường hợp của chúng ta, chúng ta muốn đo độ phân kỳ giữa xác suất mục tiêu p mà một neuron trong lớp mã hóa sẽ kích hoạt và xác suất thực tế q , được ước tính bằng cách đo kích hoạt trung bình trên batch huấn luyện. Vì vậy, độ phân kỳ KL được đơn giản hóa thành Công thức 17-2.

Công thức 17-2. Độ phân kỳ KL giữa tính thừa thớt mục tiêu p và tính thừa thớt thực tế q

$$D_{\text{KL}}(p \parallel q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}$$

Một khi chúng ta đã tính mất thừa thớt cho mỗi neuron trong lớp mã hóa, chúng ta tổng hợp các mất này và thêm kết quả vào hàm chi phí. Để kiểm soát tầm quan trọng tương đối của mất thừa thớt và mất tái tạo, chúng ta có thể nhân mất thừa thớt với một siêu tham số trọng số thừa thớt. Nếu trọng số này quá cao, mô hình sẽ bám sát tính thừa thớt mục tiêu, nhưng nó có thể không tái tạo đầu vào đúng cách, làm cho mô hình trở nên vô dụng. Ngược lại, nếu nó quá thấp, mô hình sẽ bỏ qua mục tiêu thừa thớt và sẽ không học được bất kỳ đặc trưng thú vị nào.

Bây giờ chúng ta có tất cả những gì cần thiết để triển khai một autoencoder thừa dựa trên độ phân kỳ KL. Đầu tiên, hãy tạo một bộ chuẩn hóa tùy chỉnh để áp dụng chuẩn hóa độ phân kỳ KL:

```
import tensorflow as tf

kl_divergence = tf.keras.losses.kullback_leibler_divergence

class KLDivergenceRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, weight, target):
        self.weight = weight
        self.target = target

    def __call__(self, inputs):
        mean_activities = tf.reduce_mean(inputs, axis=0)
        return self.weight * (
            kl_divergence(self.target, mean_activities) +
            kl_divergence(1. - self.target, 1. - mean_activities)
        )
```

Bây giờ chúng ta có thể xây dựng autoencoder thừa, sử dụng KLDivergenceRegularizer cho các kích hoạt của lớp mã hóa:

```
import tensorflow as tf

kld_reg = KLDivergenceRegularizer(weight=5e-3, target=0.1)
sparse_kl_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid",
                           activity_regularizer=kld_reg) # Áp dụng bộ chuẩn h
    óa KL
])
sparse_kl_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
```

```
])  
sparse_kl_ae = tf.keras.Sequential([sparse_kl_encoder, sparse_kl_decoder])
```

Sau khi huấn luyện autoencoder thừa này trên Fashion MNIST, lớp mã hóa sẽ có độ thừa thớt khoảng 10%.

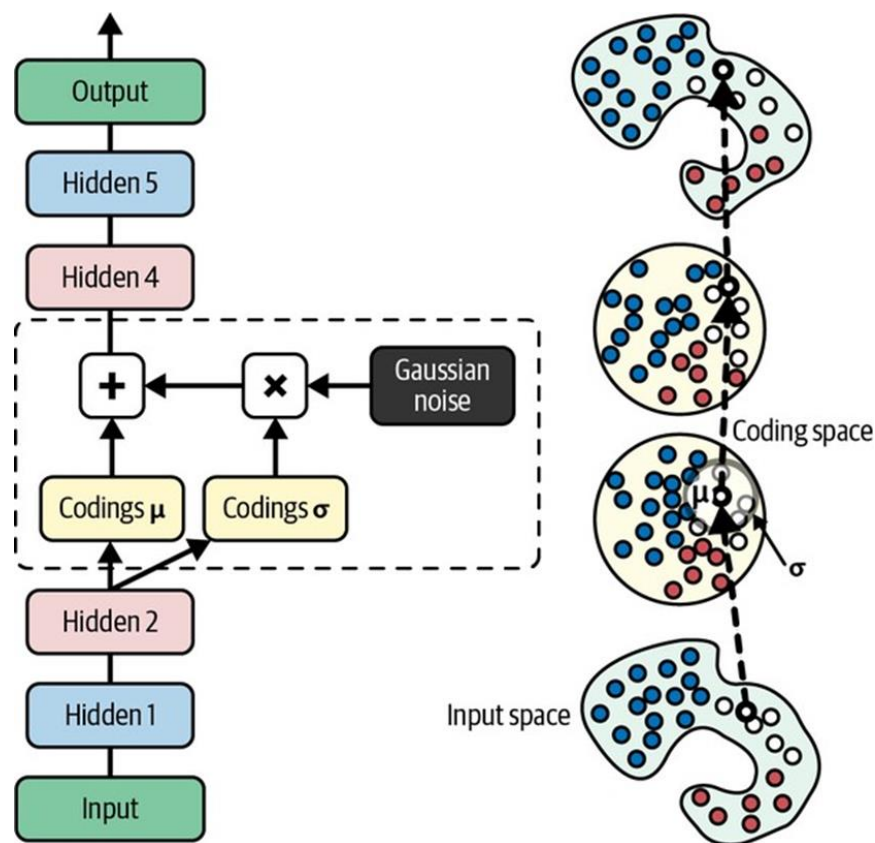
Autoencoder biến phân (Variational Autoencoders - VAEs)

Một loại autoencoder quan trọng đã được giới thiệu vào năm 2013 bởi Diederik Kingma và Max Welling và nhanh chóng trở thành một trong những biến thể phổ biến nhất: autoencoder biến phân (VAEs). VAEs khá khác biệt so với tất cả các autoencoder chúng ta đã thảo luận cho đến nay, theo những cách đặc biệt sau:

- Chúng là các autoencoder xác suất, nghĩa là đầu ra của chúng một phần được xác định bởi sự ngẫu nhiên, ngay cả sau khi huấn luyện (ngược lại với autoencoder khử nhiễu, chỉ sử dụng tính ngẫu nhiên trong quá trình huấn luyện).
- Quan trọng nhất, chúng là các autoencoder sinh, nghĩa là chúng có thể tạo ra các trường hợp mới trông giống như chúng được lấy mẫu từ tập huấn luyện.

Cả hai thuộc tính này làm cho VAEs khá giống với RBMs, nhưng chúng dễ huấn luyện hơn và quá trình lấy mẫu nhanh hơn nhiều (với RBMs bạn cần đợi mạng ổn định thành “cân bằng nhiệt” trước khi có thể lấy mẫu một trường hợp mới). Như tên gọi của chúng, autoencoder biến phân thực hiện suy luận Bayes biến phân, đây là một cách hiệu quả để thực hiện suy luận Bayes gần đúng. Nhớ lại rằng suy luận Bayes có nghĩa là cập nhật một phân phối xác suất dựa trên dữ liệu mới, sử dụng các phương trình được lấy từ định lý Bayes. Phân phối gốc được gọi là tiền định (prior), trong khi phân phối được cập nhật được gọi là hậu định (posterior). Trong trường hợp của chúng ta, chúng ta muốn tìm một xấp xỉ tốt của phân phối dữ liệu. Một khi chúng ta có điều đó, chúng ta có thể lấy mẫu từ nó.

Hãy cùng xem VAEs hoạt động như thế nào. Hình 17-11 (trái) cho thấy một autoencoder biến phân. Bạn có thể nhận ra cấu trúc cơ bản của tất cả các autoencoder, với một bộ mã hóa theo sau bởi một bộ giải mã (trong ví dụ này, cả hai đều có hai lớp ẩn), nhưng có một điểm khác biệt: thay vì trực tiếp tạo ra một mã hóa cho một đầu vào nhất định, bộ mã hóa tạo ra một mã hóa trung bình μ và một độ lệch chuẩn σ . Mã hóa thực tế sau đó được lấy mẫu ngẫu nhiên từ một phân phối Gaussian với trung bình μ và độ lệch chuẩn σ . Sau đó, bộ giải mã giải mã mã hóa đã lấy mẫu một cách bình thường. Phần bên phải của sơ đồ cho thấy một trường hợp huấn luyện đi qua autoencoder này. Đầu tiên, bộ mã hóa tạo ra μ và σ , sau đó một mã hóa được lấy mẫu ngẫu nhiên (lưu ý rằng nó không nằm chính xác tại μ), và cuối cùng mã hóa này được giải mã; đầu ra cuối cùng giống với trường hợp huấn luyện.



Hình 17-11. Một autoencoder biến phân (trái) và một trường hợp đi qua nó (phải)

Như bạn có thể thấy trong sơ đồ, mặc dù đầu vào có thể có một phân phối rất phức tạp, một autoencoder biến phân có xu hướng tạo ra các mã hóa trông như thể chúng được lấy mẫu từ một phân phối Gaussian đơn giản: trong quá trình huấn luyện, hàm chi phí (được thảo luận tiếp theo) đẩy các mã hóa dần dần di chuyển trong không gian mã hóa (còn gọi là không gian tiềm ẩn) để cuối cùng trông giống như một đám mây các điểm Gaussian. Một hệ quả tuyệt vời là sau khi huấn luyện một autoencoder biến phân, bạn có thể rất dễ dàng tạo ra một trường hợp mới: chỉ cần lấy mẫu một mã hóa ngẫu nhiên từ phân phối Gaussian, giải mã nó, và thế là xong!

Bây giờ, hãy xem xét hàm chi phí. Nó bao gồm hai phần. Phần đầu tiên là mất tái tạo thông thường đẩy autoencoder tái tạo lại đầu vào của nó. Chúng ta có thể sử dụng MSE cho việc này, như chúng ta đã làm trước đây. Phần thứ hai là mất tiềm ẩn đẩy autoencoder có các mã hóa trông như thể chúng được lấy mẫu từ một phân phối Gaussian đơn giản: đó là độ phân kỳ KL giữa phân phối mục tiêu (tức là phân phối Gaussian) và phân phối thực tế của các mã hóa. Toán học phức tạp hơn một chút so với autoencoder thưa, đặc biệt là do nhiều Gaussian, giới hạn lượng thông tin có thể được truyền đến lớp mã hóa. Điều này thúc đẩy autoencoder học các đặc trưng hữu ích. May mắn thay, các phương trình được đơn giản hóa, vì vậy mất tiềm ẩn có thể được tính bằng Công thức 17-3.

Công thức 17-3. Mất tiềm ẩn của autoencoder biến phân

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2]$$

Trong phương trình này, L là mất tiềm ẩn, n là chiều của các mã hóa, và μ_i và σ_i là trung bình và độ lệch chuẩn của thành phần thứ i của các mã hóa. Các vector μ và σ (chứa tất cả các μ_i và σ_i) được xuất bởi bộ mã hóa, như được hiển thị trong Hình 17-11 (trái).

Một điều chỉnh phổ biến cho kiến trúc của autoencoder biến phân là làm cho bộ mã hóa xuất $\gamma = \log(\sigma^2)$ thay vì σ . Mất tiềm ẩn sau đó có thể được tính như trong Công thức 17-4. Cách tiếp cận này ổn định hơn về mặt số học và tăng tốc độ huấn luyện.

Công thức 17-4. Mất tiềm ẩn của autoencoder biến phân, được viết lại bằng $\gamma = \log(\sigma^2)$

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \gamma_i - \exp(\gamma_i) - \mu_i^2]$$

Hãy bắt đầu xây dựng một autoencoder biến phân cho Fashion MNIST (như được hiển thị trong Hình 17-11, nhưng sử dụng điều chỉnh γ). Đầu tiên, chúng ta sẽ cần một lớp tùy chỉnh để lấy mẫu các mã hóa, với μ và γ :

```
import tensorflow as tf

class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs
        return tf.random.normal(tf.shape(log_var)) * tf.exp(log_var / 2) + mean
```

Lớp Sampling này nhận hai đầu vào: mean (μ) và log_var (γ). Nó sử dụng hàm `tf.random.normal()` để lấy mẫu một vector ngẫu nhiên (có cùng hình dạng với γ) từ phân phối Gaussian, với trung bình 0 và độ lệch chuẩn 1. Sau đó, nó nhân với $\exp(\gamma / 2)$ (bằng σ , như bạn có thể kiểm tra bằng toán học), và cuối cùng nó cộng μ và trả về kết quả. Điều này lấy mẫu một vector mã hóa từ phân phối Gaussian với trung bình μ và độ lệch chuẩn σ .

Tiếp theo, chúng ta có thể tạo bộ mã hóa, sử dụng API hàm vì mô hình không hoàn toàn tuần tự:

```
import tensorflow as tf

codings_size = 10

inputs = tf.keras.layers.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)
codings_mean = tf.keras.layers.Dense(codings_size)(Z) # μ
codings_log_var = tf.keras.layers.Dense(codings_size)(Z) # γ
codings = Sampling()([codings_mean, codings_log_var])
```

```
variational_encoder = tf.keras.Model(
    inputs=[inputs], outputs=[codings_mean, codings_log_var, codings])
```

Lưu ý rằng các lớp Dense xuất codings_mean (μ) và codings_log_var (γ) có cùng đầu vào (tức là đầu ra của lớp Dense thứ hai). Sau đó, chúng ta truyền cả codings_mean và codings_log_var cho lớp Sampling. Cuối cùng, mô hình $\text{variational_encoder}$ có ba đầu ra. Chỉ cần các mã hóa, nhưng chúng ta cũng thêm codings_mean và codings_log_var , phòng khi chúng ta muốn kiểm tra giá trị của chúng. Bây giờ hãy xây dựng bộ giải mã:

```
import tensorflow as tf

decoder_inputs = tf.keras.layers.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="relu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="relu")(x)
x = tf.keras.layers.Dense(28 * 28)(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)
variational_decoder = tf.keras.Model(inputs=[decoder_inputs], outputs=[outputs])
```

Đối với bộ giải mã này, chúng ta có thể đã sử dụng API tuần tự thay vì API hàm, vì nó thực sự chỉ là một chồng lớp đơn giản, gần như giống hệt với nhiều bộ giải mã chúng ta đã xây dựng cho đến nay. Cuối cùng, hãy xây dựng mô hình autoencoder biến phân:

```
import tensorflow as tf

# Đảm bảo inputs đã được định nghĩa từ phần variational_encoder
# _, _, codings = variational_encoder(inputs) # Lấy codings từ encoder đã tạo
reconstructions = variational_decoder(codings)
variational_ae = tf.keras.Model(inputs=[inputs], outputs=[reconstructions])
```

Chúng ta bỏ qua hai đầu ra đầu tiên của bộ mã hóa (chúng ta chỉ muốn cấp các mã hóa cho bộ giải mã). Cuối cùng, chúng ta phải thêm mất tiềm ẩn và mất tái tạo:

```
import tensorflow as tf

latent_loss = -0.5 * tf.reduce_sum(
    1 + codings_log_var - tf.exp(codings_log_var) - tf.square(codings_mean),
    axis=-1)
variational_ae.add_loss(tf.reduce_mean(latent_loss) / 784.)
```

Chúng ta đầu tiên áp dụng Công thức 17-4 để tính mất tiềm ẩn cho mỗi trường hợp trong batch, tổng hợp trên trục cuối cùng. Sau đó, chúng ta tính mất trung bình trên tất cả các trường hợp trong batch, và chúng ta chia kết quả cho 784 để đảm bảo nó có tỷ lệ thích hợp so với mất tái tạo. Thật vậy, mất tái tạo của autoencoder biến phân được cho là tổng các lỗi tái tạo pixel, nhưng khi Keras tính mất “mse”, nó tính giá trị trung bình trên tất cả 784 pixel, thay vì tổng. Vì vậy, mất tái tạo nhỏ hơn 784 lần so với mức chúng ta cần. Chúng ta có thể định nghĩa một hàm mất tùy chỉnh để tính tổng thay vì trung bình, nhưng đơn giản hơn là chia mất tiềm ẩn cho 784 (tổng mất cuối cùng sẽ nhỏ hơn 784 lần so với đáng lẽ, nhưng điều này chỉ có nghĩa là chúng ta nên sử dụng tốc độ học lớn hơn).

Và cuối cùng, chúng ta có thể biên dịch và huấn luyện autoencoder!

```
import tensorflow as tf

# Giả định X_train và X_valid đã được định nghĩa từ ngữ cảnh trước đó
variational_ae.compile(loss="mse", optimizer="nadam")
# history = variational_ae.fit(X_train, X_train, epochs=25, batch_size=128,
#                               validation_data=(X_valid, X_valid))
```

Tạo ảnh Fashion MNIST

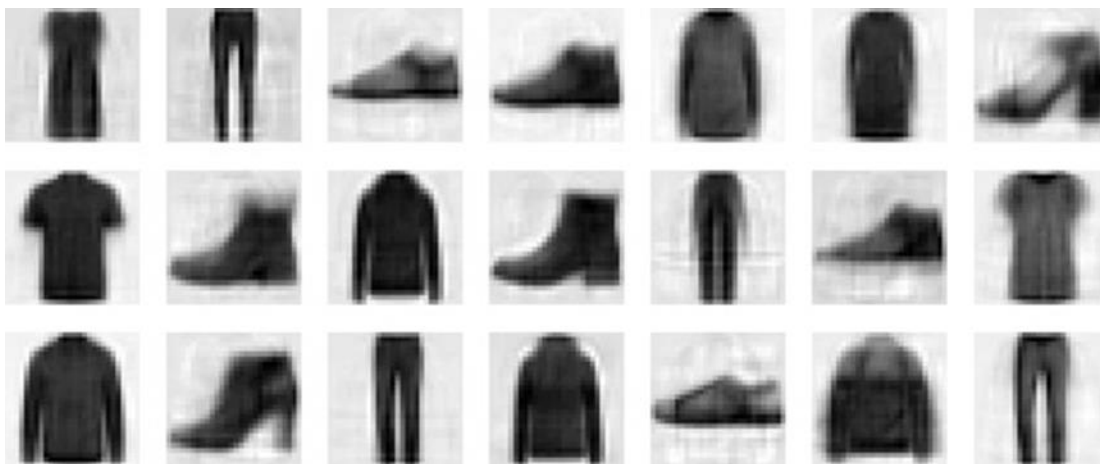
Bây giờ chúng ta hãy sử dụng autoencoder biến phân này để tạo ra những hình ảnh trông giống các mặt hàng thời trang. Tất cả những gì chúng ta cần làm là lấy mẫu các mã hóa ngẫu nhiên từ một phân phối Gaussian và giải mã chúng:

```
import tensorflow as tf
import numpy as np # Đảm bảo import numpy nếu chưa có

# Giả định codings_size đã được định nghĩa là 10 từ phần Variational Autoencoders trước đó
# Giả định variational_decoder đã được xây dựng và có sẵn từ ngữ cảnh trước đó

# codings = tf.random.normal(shape=[3 * 7, codings_size])
# images = variational_decoder(codings).numpy()
```

Hình 17-12 cho thấy 12 hình ảnh được tạo ra.



Hình 17-12. Hình ảnh Fashion MNIST được tạo bởi autoencoder biến phân

Phần lớn các hình ảnh này trông khá thuyết phục, mặc dù hơi mờ. Phần còn lại thì không đẹp lắm, nhưng đừng quá khắc nghiệt với autoencoder—nó chỉ có vài phút để học!

Autoencoder biến phân giúp có thể thực hiện phép nội suy ngữ nghĩa: thay vì nội suy giữa hai hình ảnh ở cấp độ pixel, điều này sẽ trông như thể hai hình ảnh chỉ được phủ lên nhau, chúng ta có thể nội suy ở cấp độ mã hóa. Ví dụ, chúng ta hãy lấy một vài mã hóa dọc theo một đường

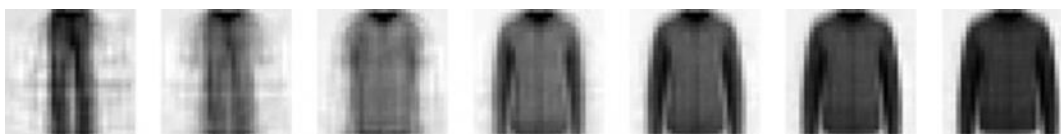
tùy ý trong không gian tiềm ẩn và giải mã chúng. Chúng ta sẽ có một chuỗi hình ảnh dần dần chuyển từ quần sang áo len (xem Hình 17-13):

```
import numpy as np
import matplotlib.pyplot as plt # Đảm bảo import matplotlib.pyplot nếu chưa có

# Giả định codings_size đã được định nghĩa là 10 từ phần Variational Autoencoders trước đó
# Giả định variational_decoder đã được xây dựng và có sẵn từ ngữ cảnh trước đó

# codings = np.zeros([7, codings_size])
# codings[:, 3] = np.linspace(-0.8, 0.8, 7) #trục 3 trông tốt nhất trong trường hợp này
# images = variational_decoder(codings).numpy()

# Để hiển thị các hình ảnh nội suy, bạn sẽ cần một hàm hiển thị tương tự plot_reconstructions
# hoặc vòng lặp để hiển thị từng hình ảnh.
# Ví dụ:
# fig = plt.figure(figsize=(7 * 1.5, 3))
# for i in range(7):
#     plt.subplot(1, 7, 1 + i)
#     plt.imshow(images[i], cmap="binary")
#     plt.axis("off")
# plt.show()
```



Hình 17-13. Nội suy ngữ nghĩa

Bây giờ chúng ta hãy chuyển sự chú ý sang GAN: chúng khó huấn luyện hơn, nhưng khi bạn xoay sở để làm cho chúng hoạt động, chúng tạo ra những hình ảnh khá tuyệt vời.

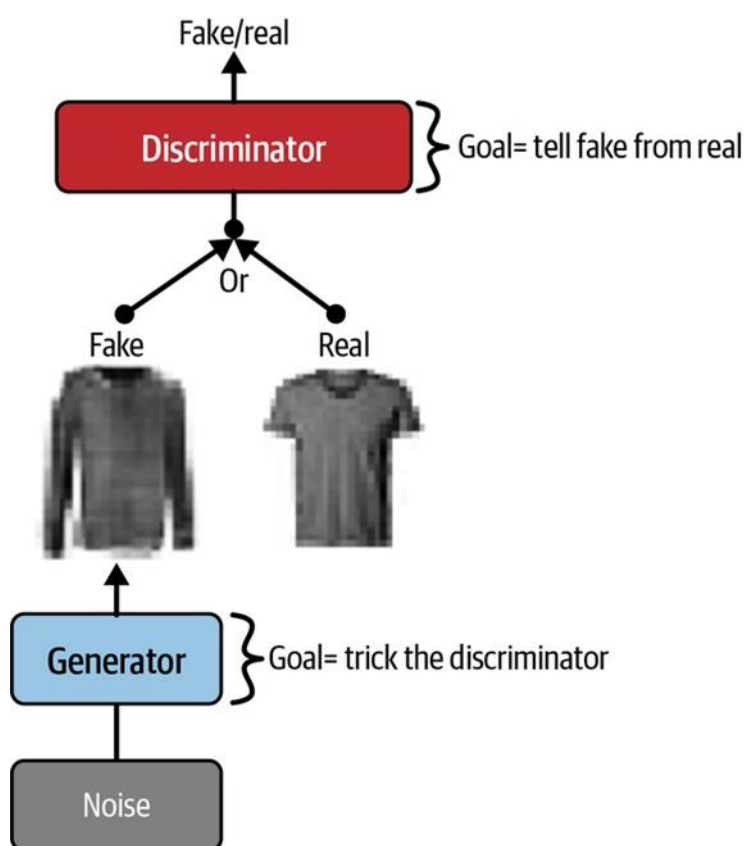
Mạng đối kháng sinh (Generative Adversarial Networks - GANs)

Mạng đối kháng sinh được đề xuất trong một bài báo năm 2014 bởi Ian Goodfellow và cộng sự, và mặc dù ý tưởng này đã khiến các nhà nghiên cứu hứng thú gần như ngay lập tức, phải mất vài năm để khắc phục một số khó khăn trong việc huấn luyện GAN. Giống như nhiều ý tưởng tuyệt vời, nó có vẻ đơn giản khi nhìn lại: làm cho các mạng thần kinh cạnh tranh với nhau với hy vọng rằng sự cạnh tranh này sẽ thúc đẩy chúng vượt trội. Như được hiển thị trong Hình 17-14, một GAN bao gồm hai mạng thần kinh:

- **Bộ tạo (Generator):** Lấy một phân phối ngẫu nhiên làm đầu vào (thường là Gaussian) và xuất ra một số dữ liệu — điển hình là một hình ảnh. Bạn có thể coi các đầu vào

ngẫu nhiên là các biểu diễn tiềm ẩn (tức là mã hóa) của hình ảnh sẽ được tạo ra. Vì vậy, như bạn có thể thấy, bộ tạo cung cấp chức năng tương tự như một bộ giải mã trong autoencoder biến phân, và nó có thể được sử dụng theo cùng một cách để tạo ra hình ảnh mới: chỉ cần cấp cho nó một số nhiễu Gaussian, và nó xuất ra một hình ảnh hoàn toàn mới. Tuy nhiên, nó được huấn luyện rất khác, như bạn sẽ sớm thấy.

- **Bộ phân biệt (Discriminator):** Lấy một hình ảnh giả từ bộ tạo hoặc một hình ảnh thật từ tập huấn luyện làm đầu vào, và phải đoán xem hình ảnh đầu vào là giả hay thật.



Hình 17-14. Một mạng đối kháng sinh

Trong quá trình huấn luyện, bộ tạo và bộ phân biệt có các mục tiêu đối lập: bộ phân biệt cố gắng phân biệt hình ảnh giả với hình ảnh thật, trong khi bộ tạo cố gắng tạo ra những hình ảnh trông đủ thật để đánh lừa bộ phân biệt.

Vì GAN bao gồm hai mạng với các mục tiêu khác nhau, nó không thể được huấn luyện như một mạng thần kinh thông thường. Mỗi lần lặp huấn luyện được chia thành hai giai đoạn:

- **Trong giai đoạn đầu tiên,** chúng ta huấn luyện bộ phân biệt. Một batch các hình ảnh thật được lấy mẫu từ tập huấn luyện và được bổ sung bằng một số lượng hình ảnh giả tương đương được tạo bởi bộ tạo. Các nhãn được đặt là 0 cho hình ảnh giả và 1 cho hình ảnh thật, và bộ phân biệt được huấn luyện trên batch được gán nhãn này trong một bước, sử dụng hàm mất entropy chéo nhị phân. Quan trọng là, lan truyền ngược chỉ tối ưu hóa trọng số của bộ phân biệt trong giai đoạn này.

- **Trong giai đoạn thứ hai**, chúng ta huấn luyện bộ tạo. Đầu tiên chúng ta sử dụng nó để tạo ra một batch hình ảnh giả khác, và một lần nữa bộ phân biệt được sử dụng để phân biệt xem hình ảnh là giả hay thật. Lần này chúng ta không thêm hình ảnh thật vào batch, và tất cả các nhãn được đặt là 1 (thật): nói cách khác, chúng ta muốn bộ tạo tạo ra những hình ảnh mà bộ phân biệt sẽ (sai lầm) tin là thật! Quan trọng là, trọng số của bộ phân biệt bị đóng băng trong bước này, vì vậy lan truyền ngược chỉ ảnh hưởng đến trọng số của bộ tạo.

Hãy tiến hành xây dựng một GAN đơn giản cho Fashion MNIST.

Đầu tiên, chúng ta cần xây dựng bộ tạo và bộ phân biệt. Bộ tạo tương tự như bộ giải mã của autoencoder, và bộ phân biệt là một bộ phân loại nhị phân thông thường: nó nhận một hình ảnh làm đầu vào và kết thúc bằng một lớp Dense chứa một đơn vị duy nhất và sử dụng hàm kích hoạt sigmoid. Đối với giai đoạn thứ hai của mỗi lần lặp huấn luyện, chúng ta cũng cần mô hình GAN đầy đủ chứa bộ tạo theo sau bởi bộ phân biệt:

```
import tensorflow as tf

codings_size = 30

Dense = tf.keras.layers.Dense # Đặt một biến ngắn gọn cho lớp Dense
generator = tf.keras.Sequential([
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])
discriminator = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(1, activation="sigmoid") # Đầu ra nhị phân (thật/giả)
])
gan = tf.keras.Sequential([generator, discriminator])
```

Tiếp theo, chúng ta cần biên dịch các mô hình này. Vì bộ phân biệt là một bộ phân loại nhị phân, chúng ta có thể tự nhiên sử dụng hàm mất entropy chéo nhị phân. Mô hình gan cũng là một bộ phân loại nhị phân, vì vậy nó cũng có thể sử dụng hàm mất entropy chéo nhị phân.

Tuy nhiên, bộ tạo sẽ chỉ được huấn luyện thông qua mô hình gan, vì vậy chúng ta không cần biên dịch nó. Quan trọng là, bộ phân biệt không nên được huấn luyện trong giai đoạn thứ hai, vì vậy chúng ta đặt nó không thể huấn luyện được trước khi biên dịch mô hình gan:

```
import tensorflow as tf

# Giả định discriminator và gan đã được định nghĩa từ mã trước đó
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

```
discriminator.trainable = False # Đóng bằng trọng số của discriminator
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

Vì vòng lặp huấn luyện bất thường, chúng ta không thể sử dụng phương thức `fit()` thông thường. Thay vào đó, chúng ta sẽ viết một vòng lặp huấn luyện tùy chỉnh. Đối với điều này, chúng ta cần tạo một Dataset để lặp qua các hình ảnh:

```
import tensorflow as tf
# Giả định X_train đã được định nghĩa và có sẵn từ ngữ cảnh trước đó

batch_size = 32
dataset = tf.data.Dataset.from_tensor_slices(X_train).shuffle(buffer_size=1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

Bây giờ chúng ta đã sẵn sàng để viết vòng lặp huấn luyện. Hãy gói nó trong một hàm `train_gan()`:

```
import tensorflow as tf
import numpy as np # Đảm bảo import numpy nếu chưa có

def train_gan(gan, dataset, batch_size, codings_size, n_epochs):
    generator, discriminator = gan.layers

    for epoch in range(n_epochs):
        print(f"Epoch {epoch+1}/{n_epochs}")
        for X_batch in dataset:
            # Giai đoạn 1 - huấn luyện bộ phân biệt
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = generator(noise)
            X_fake_and_real = tf.concat([generated_images, X_batch], axis=0)
            # Nhãn: 0 cho giả, 1 cho thật
            y1 = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)
            discriminator.train_on_batch(X_fake_and_real, y1)

            # Giai đoạn 2 - huấn luyện bộ tạo
            noise = tf.random.normal(shape=[batch_size, codings_size])
            # Nhãn: tất cả là 1 (muốn bộ phân biệt nghĩ là thật)
            y2 = tf.constant([[1.]] * batch_size)
            gan.train_on_batch(noise, y2)

# Giả định gan, dataset, batch_size, codings_size đã được định nghĩa
# train_gan(gan, dataset, batch_size, codings_size, n_epochs=50)
```

Như đã thảo luận trước đó, bạn có thể thấy hai giai đoạn ở mỗi lần lặp:

- **Trong giai đoạn một**, chúng ta cấp nhiễu Gaussian cho bộ tạo để tạo ra hình ảnh giả, và chúng ta bổ sung batch này bằng cách nối một số lượng hình ảnh thật tương đương. Các mục tiêu `y1` được đặt là 0 cho hình ảnh giả và 1 cho hình ảnh thật. Sau đó, chúng

ta huấn luyện bộ phân biệt trên batch này. Hãy nhớ rằng bộ phân biệt có thể huấn luyện được trong giai đoạn này, nhưng chúng ta không động đến bộ tạo.

- **Trong giai đoạn hai**, chúng ta cấp cho GAN một số nhiễu Gaussian. Bộ tạo của nó sẽ bắt đầu bằng cách tạo ra hình ảnh giả, sau đó bộ phân biệt sẽ cố gắng đoán xem những hình ảnh này là giả hay thật. Trong giai đoạn này, chúng ta đang cố gắng cải thiện bộ tạo, điều đó có nghĩa là chúng ta muốn bộ phân biệt thất bại: đây là lý do tại sao các mục tiêu y_2 đều được đặt là 1, mặc dù hình ảnh là giả. Trong giai đoạn này, bộ phân biệt không thể huấn luyện được, vì vậy phần duy nhất của mô hình gan sẽ cải thiện là bộ tạo.

Vậy đó! Sau khi huấn luyện, bạn có thể lấy mẫu ngẫu nhiên một số mã hóa từ phân phối Gaussian, và cấp chúng cho bộ tạo để tạo ra hình ảnh mới:

```
import tensorflow as tf
# Giả định batch_size, codings_size đã được định nghĩa
# Giả định generator đã được huấn luyện và có sẵn

# codings = tf.random.normal(shape=[batch_size, codings_size])
# generated_images = generator.predict(codings)
```

Nếu bạn hiển thị các hình ảnh được tạo (xem Hình 17-15), bạn sẽ thấy rằng ở cuối epoch đầu tiên, chúng đã bắt đầu trông giống (rất nhiều) hình ảnh Fashion MNIST.



Hình 17-15. Hình ảnh được tạo bởi GAN sau một epoch huấn luyện

Thật không may, các hình ảnh không bao giờ thực sự tốt hơn nhiều so với thế, và bạn thậm chí có thể thấy các epoch mà GAN dường như đang quên những gì nó đã học. Tại sao lại như vậy? Chà, hóa ra việc huấn luyện một GAN có thể rất khó khăn. Hãy xem tại sao.

Những khó khăn khi huấn luyện GAN

Trong quá trình huấn luyện, bộ tạo và bộ phân biệt liên tục cố gắng vượt trội lẫn nhau, trong một trò chơi tổng bằng không. Khi quá trình huấn luyện tiến triển, trò chơi có thể kết thúc ở

trạng thái mà các nhà lý thuyết trò chơi gọi là cân bằng Nash, được đặt tên theo nhà toán học John Nash: đây là khi không người chơi nào tốt hơn nếu thay đổi chiến lược của riêng họ, giả sử những người chơi khác không thay đổi của họ. Ví dụ, cân bằng Nash đạt được khi mọi người lái xe bên trái đường: không người lái nào tốt hơn nếu là người duy nhất chuyển làn. Tất nhiên, có một cân bằng Nash thứ hai có thể xảy ra: khi mọi người lái xe bên phải đường. Các trạng thái và động lực ban đầu khác nhau có thể dẫn đến cân bằng này hay cân bằng khác. Trong ví dụ này, có một chiến lược tối ưu duy nhất một khi cân bằng được đạt đến (tức là lái xe cùng phía với mọi người khác), nhưng cân bằng Nash có thể liên quan đến nhiều chiến lược cạnh tranh (ví dụ: một kẻ săn mồi đuổi theo con mồi, con mồi cố gắng thoát thân, và cả hai đều không tốt hơn nếu thay đổi chiến lược của họ).

Vậy điều này áp dụng cho GAN như thế nào? Chà, các tác giả của bài báo GAN đã chứng minh rằng một GAN chỉ có thể đạt được một cân bằng Nash duy nhất: đó là khi bộ tạo tạo ra hình ảnh hoàn toàn chân thực, và bộ phân biệt bị buộc phải đoán (50% thật, 50% giả). Thực tế này rất đáng khích lệ: dường như bạn chỉ cần huấn luyện GAN đủ lâu, và nó cuối cùng sẽ đạt đến cân bằng này, mang lại cho bạn một bộ tạo hoàn hảo. Thật không may, điều đó không đơn giản: không có gì đảm bảo rằng cân bằng sẽ đạt được.

Khó khăn lớn nhất được gọi là **sập chế độ (mode collapse)**: đây là khi đầu ra của bộ tạo dần trở nên kém đa dạng hơn. Điều này có thể xảy ra như thế nào? Giả sử bộ tạo giỏi tạo ra những đôi giày thuyết phục hơn bất kỳ lớp nào khác. Nó sẽ đánh lừa bộ phân biệt nhiều hơn một chút với giày, và điều này sẽ khuyến khích nó tạo ra nhiều hình ảnh giày hơn nữa. Dần dần, nó sẽ quên cách tạo ra bất cứ thứ gì khác. Trong khi đó, những hình ảnh giả duy nhất mà bộ phân biệt sẽ thấy là giày, vì vậy nó cũng sẽ quên cách phân biệt hình ảnh giả của các lớp khác. Cuối cùng, khi bộ phân biệt quản lý phân biệt giày giả với giày thật, bộ tạo sẽ bị buộc phải chuyển sang một lớp khác. Sau đó, nó có thể trở nên giỏi về áo sơ mi, quên giày, và bộ phân biệt sẽ theo sau. GAN có thể dần dần luân chuyển qua một vài lớp, không bao giờ thực sự trở nên rất giỏi về bất kỳ lớp nào trong số đó.

Hơn nữa, vì bộ tạo và bộ phân biệt liên tục đẩy nhau, các tham số của chúng có thể kết thúc dao động và trở nên không ổn định. Quá trình huấn luyện có thể bắt đầu đúng cách, sau đó đột nhiên phân kỳ không rõ lý do, do những bất ổn này. Và vì nhiều yếu tố ảnh hưởng đến các động lực phức tạp này, GAN rất nhạy cảm với các siêu tham số: bạn có thể phải tốn nhiều công sức để tinh chỉnh chúng. Trên thực tế, đó là lý do tại sao tôi sử dụng RMSProp thay vì Nadam khi biên dịch các mô hình: khi sử dụng Nadam, tôi đã gặp phải sự sụp đổ nghiêm trọng.

Những vấn đề này đã khiến các nhà nghiên cứu rất bận rộn kể từ năm 2014: nhiều bài báo đã được xuất bản về chủ đề này, một số đề xuất các hàm chi phí mới (mặc dù một bài báo năm 2018 của các nhà nghiên cứu Google đặt câu hỏi về hiệu quả của chúng) hoặc các kỹ thuật để ổn định huấn luyện hoặc để tránh vấn đề sụp đổ chế độ. Ví dụ, một kỹ thuật phổ biến gọi là **replay kinh nghiệm (experience replay)** bao gồm việc lưu trữ các hình ảnh được tạo bởi bộ tạo ở mỗi lần lặp trong một bộ đệm phát lại (dần dần loại bỏ các hình ảnh được tạo cũ hơn) và huấn luyện bộ phân biệt bằng cách sử dụng hình ảnh thật cộng với hình ảnh giả được lấy từ bộ đệm này (thay vì chỉ các hình ảnh giả được tạo bởi bộ tạo hiện tại). Điều này làm giảm

khả năng bộ phân biệt sẽ quá khớp với đầu ra của bộ tạo mới nhất. Một kỹ thuật phổ biến khác được gọi là **phân biệt theo mini-batch (mini-batch discrimination)**: nó đo lường mức độ tương tự của hình ảnh trong toàn bộ batch và cung cấp thống kê này cho bộ phân biệt, để nó có thể dễ dàng từ chối toàn bộ batch hình ảnh giả thiếu đa dạng. Điều này khuyến khích bộ tạo tạo ra nhiều loại hình ảnh hơn, giảm khả năng sập chế độ. Các bài báo khác đơn giản đề xuất các kiến trúc cụ thể có hiệu suất tốt.

Tóm lại, đây vẫn là một lĩnh vực nghiên cứu rất năng động, và động lực của GAN vẫn chưa được hiểu hoàn hảo. Nhưng tin tốt là đã có những tiến bộ lớn, và một số kết quả thực sự đáng kinh ngạc! Vì vậy, chúng ta hãy xem xét một số kiến trúc thành công nhất, bắt đầu với GAN tích chập sâu, vốn là công nghệ tiên tiến chỉ vài năm trước. Sau đó, chúng ta sẽ xem xét hai kiến trúc gần đây hơn (và phức tạp hơn).

GAN tích chập sâu (Deep Convolutional GANs - DCGANs)

Các tác giả của bài báo GAN gốc đã thử nghiệm với các lớp tích chập, nhưng chỉ cố gắng tạo ra các hình ảnh nhỏ. Ngay sau đó, nhiều nhà nghiên cứu đã cố gắng xây dựng GAN dựa trên các mạng tích chập sâu hơn để tạo ra các hình ảnh lớn hơn.

Điều này tỏ ra khó khăn, vì quá trình huấn luyện rất không ổn định, nhưng Alec Radford và cộng sự cuối cùng đã thành công vào cuối năm 2015, sau khi thử nghiệm với nhiều kiến trúc và siêu tham số khác nhau. Họ gọi kiến trúc của mình là GAN tích chập sâu (DCGANs). Dưới đây là các hướng dẫn chính mà họ đề xuất để xây dựng các GAN tích chập ổn định:

- **Thay thế bất kỳ lớp gộp nào bằng tích chập bước (strided convolutions)** (trong bộ phân biệt) và **tích chập chuyển vị (transposed convolutions)** (trong bộ tạo).
- **Sử dụng chuẩn hóa batch (batch normalization)** trong cả bộ tạo và bộ phân biệt, ngoại trừ lớp đầu ra của bộ tạo và lớp đầu vào của bộ phân biệt.
- **Loại bỏ các lớp ẩn kết nối đầy đủ** cho các kiến trúc sâu hơn.
- **Sử dụng kích hoạt ReLU** trong bộ tạo cho tất cả các lớp ngoại trừ lớp đầu ra, lớp này nên sử dụng tanh.
- **Sử dụng kích hoạt Leaky ReLU** trong bộ phân biệt cho tất cả các lớp.

Những hướng dẫn này sẽ hoạt động trong nhiều trường hợp, nhưng không phải lúc nào cũng vậy, vì vậy bạn vẫn có thể cần thử nghiệm với các siêu tham số khác nhau. Trên thực tế, chỉ cần thay đổi hạt ngẫu nhiên và huấn luyện lại cùng một mô hình chính xác đôi khi sẽ có tác dụng. Dưới đây là một DCGAN nhỏ hoạt động khá tốt với Fashion MNIST:

```
import tensorflow as tf

codings_size = 100

generator = tf.keras.Sequential([
    tf.keras.layers.Dense(7 * 7 * 128),
    tf.keras.layers.Reshape([7, 7, 128]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2,
```

```

padding="same", activation="relu"),
tf.keras.layers.BatchNormalization(),
tf.keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2,
padding="same", activation="tanh"),
])
discriminator = tf.keras.Sequential([
tf.keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="same",
activation=tf.keras.layers.LeakyReLU(0.2)),
tf.keras.layers.Dropout(0.4),
tf.keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="same",
activation=tf.keras.layers.LeakyReLU(0.2)),
tf.keras.layers.Dropout(0.4),
tf.keras.layers.Flatten(),
tf.keras.layers.Dense(1, activation="sigmoid")
])
gan = tf.keras.Sequential([generator, discriminator])

```

Bộ tạo nhận các mã hóa có kích thước 100, chiếu chúng lên 6.272 chiều ($7 \times 7 \times 128$), và định hình lại kết quả để có một tensor $7 \times 7 \times 128$. Tensor này được chuẩn hóa batch và cấp cho một lớp tích chập chuyển vị với bước nhảy 2, lớp này sẽ tăng mẫu từ 7×7 lên 14×14 và giảm chiều sâu từ 128 xuống 64. Kết quả được chuẩn hóa batch lại và cấp cho một lớp tích chập chuyển vị khác với bước nhảy 2, lớp này sẽ tăng mẫu từ 14×14 lên 28×28 và giảm chiều sâu từ 64 xuống 1. Lớp này sử dụng hàm kích hoạt tanh, vì vậy đầu ra sẽ nằm trong khoảng từ -1 đến 1. Vì lý do này, trước khi huấn luyện GAN, chúng ta cần thay đổi tỷ lệ tập huấn luyện về cùng một khoảng này. Chúng ta cũng cần định hình lại nó để thêm chiều kênh:

```

import numpy as np
# Giả định X_train đã được định nghĩa và có sẵn từ ngữ cảnh trước đó
# X_train_dcgan = X_train.reshape(-1, 28, 28, 1) * 2. - 1. # định hình lại và
thay đổi tỷ lệ

```

Bộ phân biệt trông rất giống một CNN thông thường để phân loại nhị phân, ngoại trừ thay vì sử dụng các lớp gộp cực đại để giảm mẫu hình ảnh, chúng ta sử dụng các tích chập bước (strides=2). Lưu ý rằng chúng ta sử dụng hàm kích hoạt Leaky ReLU. Nhìn chung, chúng ta đã tuân thủ các hướng dẫn của DCGAN, ngoại trừ việc chúng ta đã thay thế các lớp BatchNormalization trong bộ phân biệt bằng các lớp Dropout; nếu không, quá trình huấn luyện trong trường hợp này không ổn định. Hãy thoải mái điều chỉnh kiến trúc này: bạn sẽ thấy nó nhạy cảm như thế nào với các siêu tham số, đặc biệt là tốc độ học tương đối của hai mạng.

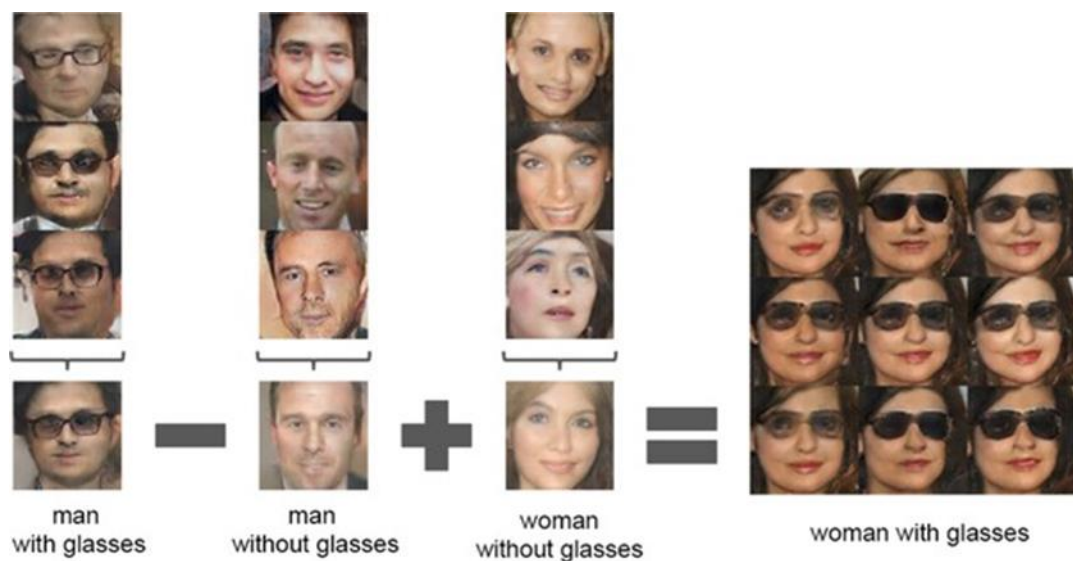
Cuối cùng, để xây dựng tập dữ liệu và sau đó biên dịch và huấn luyện mô hình này, chúng ta có thể sử dụng cùng một mã như trước đó. Sau 50 epoch huấn luyện, bộ tạo tạo ra các hình ảnh như được hiển thị trong Hình 17-16. Nó vẫn chưa hoàn hảo, nhưng nhiều hình ảnh này khá thuyết phục.



Hình 17-16. Hình ảnh được tạo bởi DCGAN sau 50 epoch huấn luyện

Nếu bạn mở rộng kiến trúc này và huấn luyện nó trên một tập dữ liệu lớn về khuôn mặt, bạn có thể nhận được những hình ảnh khá chân thực. Trên thực tế, DCGANs có thể học các biểu diễn tiềm ẩn khá có ý nghĩa, như bạn có thể thấy trong Hình 17-17: nhiều hình ảnh đã được tạo ra, và chín trong số đó được chọn thủ công (góc trên bên trái), bao gồm ba hình đại diện cho đàn ông đeo kính, ba người đàn ông không đeo kính và ba phụ nữ không đeo kính. Đối với mỗi loại này, các mã hóa được sử dụng để tạo ra các hình ảnh đã được tính trung bình, và một hình ảnh được tạo ra dựa trên các mã hóa trung bình kết quả (phía dưới bên trái). Tóm lại, mỗi trong ba hình ảnh phía dưới bên trái đại diện cho giá trị trung bình của ba hình ảnh nằm phía trên nó. Nhưng đây không phải là một giá trị trung bình đơn giản được tính ở cấp độ pixel (điều này sẽ dẫn đến ba khuôn mặt chồng lên nhau), đó là một giá trị trung bình được tính trong không gian tiềm ẩn, vì vậy các hình ảnh vẫn trông giống như khuôn mặt bình thường. Thật đáng kinh ngạc, nếu bạn tính đàn ông đeo kính, trừ đàn ông không đeo kính, cộng với phụ nữ không đeo kính — trong đó mỗi thuật ngữ tương ứng với một trong các mã hóa trung bình — và bạn tạo ra hình ảnh tương ứng với mã hóa này, bạn sẽ nhận được hình ảnh ở trung tâm của lưới 3×3 khuôn mặt bên phải: một người phụ nữ đeo kính! Tám hình ảnh khác xung quanh nó được tạo ra dựa trên cùng một vector cộng với một chút nhiễu, để minh họa khả năng nội suy ngữ nghĩa của DCGANs. Có thể thực hiện các phép tính số học trên khuôn mặt cảm giác giống như khoa học viễn tưởng!

Tuy nhiên, DCGANs không hoàn hảo. Ví dụ, khi bạn cố gắng tạo ra các hình ảnh rất lớn bằng DCGANs, bạn thường kết thúc với các đặc trưng cục bộ thuyết phục nhưng tổng thể không nhất quán, chẳng hạn như áo sơ mi có một ống tay dài hơn nhiều so với ống kia, hoa tai khác nhau, hoặc mắt nhìn theo các hướng đối diện. Làm thế nào bạn có thể khắc phục điều này?

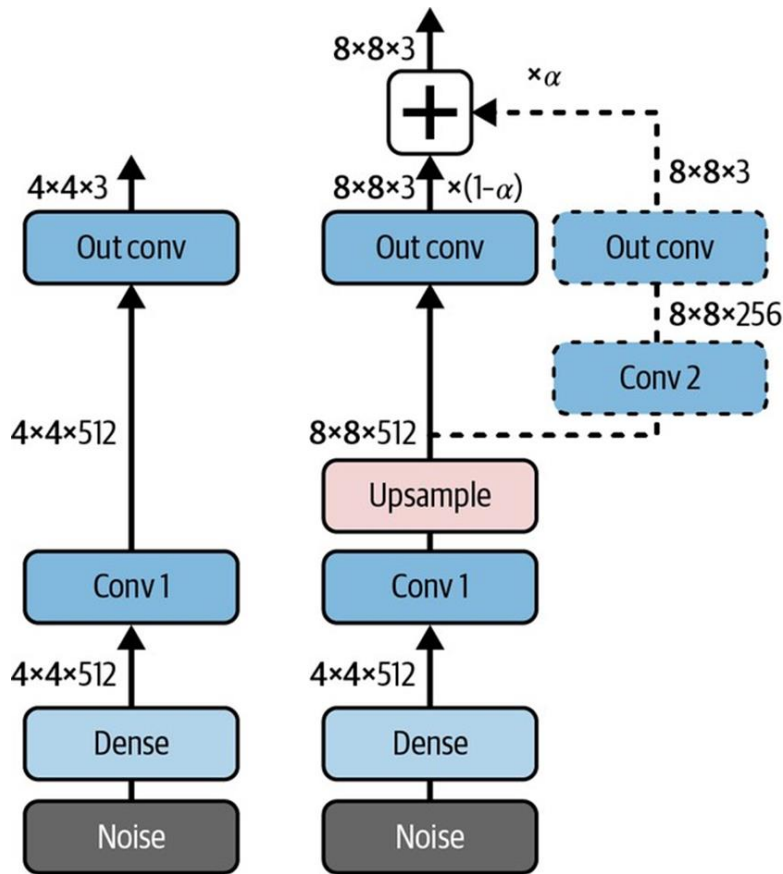


Hình 17-17. Phép toán vector cho các khái niệm hình ảnh (một phần của hình 7 từ bài báo DCGAN)

Sự phát triển tăng dần của GANs (Progressive Growing of GANs)

Trong một bài báo năm 2018, các nhà nghiên cứu của Nvidia Tero Karras và cộng sự đã đề xuất một kỹ thuật quan trọng: họ gợi ý tạo ra các hình ảnh nhỏ ở đầu quá trình huấn luyện, sau đó dần dần thêm các lớp tích chập vào cả bộ tạo và bộ phân biệt để tạo ra các hình ảnh ngày càng lớn hơn (4×4 , 8×8 , 16×16 , ..., 512×512 , 1.024×1.024). Cách tiếp cận này giống với việc huấn luyện từng lớp tham lam của các autoencoder xếp chồng. Các lớp bổ sung được thêm vào cuối bộ tạo và ở đầu bộ phân biệt, và các lớp đã được huấn luyện trước đó vẫn có thể huấn luyện được.

Ví dụ, khi tăng kích thước đầu ra của bộ tạo từ 4×4 lên 8×8 (xem Hình 17-18), một lớp tăng mẫu (sử dụng lọc lân cận gần nhất) được thêm vào lớp tích chập hiện có (“Conv 1”) để tạo ra các bản đồ đặc trưng 8×8 . Những bản đồ này được cấp cho lớp tích chập mới (“Conv 2”), lớp này lần lượt cấp vào một lớp tích chập đầu ra mới. Để tránh làm hỏng các trọng số đã được huấn luyện của Conv 1, chúng ta dần dần làm mờ hai lớp tích chập mới (được biểu thị bằng đường đứt nét trong Hình 17-18) và làm mờ lớp đầu ra gốc. Đầu ra cuối cùng là tổng có trọng số của các đầu ra mới (với trọng số α) và các đầu ra gốc (với trọng số $1 - \alpha$), từ từ tăng α từ 0 đến 1. Một kỹ thuật làm mờ tương tự được sử dụng khi một lớp tích chập mới được thêm vào bộ phân biệt (theo sau là một lớp gộp trung bình để giảm mẫu). Lưu ý rằng tất cả các lớp tích chập đều sử dụng padding “same” và bước nhảy 1, vì vậy chúng bảo toàn chiều cao và chiều rộng của đầu vào. Điều này bao gồm lớp tích chập gốc, vì vậy giờ đây nó tạo ra đầu ra 8×8 (vì đầu vào của nó bây giờ là 8×8). Cuối cùng, các lớp đầu ra sử dụng kích thước kernel 1. Chúng chỉ chiếu đầu vào của chúng xuống số lượng kênh màu mong muốn (thường là 3).



Hình 17-18. Một GAN phát triển dần dần: bộ tạo GAN xuất ra hình ảnh màu 4×4 (trái); chúng tôi mở rộng nó để xuất ra hình ảnh 8×8 (phải)

Bài báo cũng giới thiệu một số kỹ thuật khác nhằm tăng tính đa dạng của đầu ra (để tránh sụp chế độ) và làm cho quá trình huấn luyện ổn định hơn:

- Lớp độ lệch chuẩn mini-batch (Mini-batch standard deviation layer):** Được thêm gần cuối bộ phân biệt. Đối với mỗi vị trí trong đầu vào, nó tính độ lệch chuẩn trên tất cả các kênh và tất cả các trường hợp trong batch ($S = \text{tf.math.reduce_std}(\text{inputs}, \text{axis}=[0, -1])$). Các độ lệch chuẩn này sau đó được tính trung bình trên tất cả các điểm để có được một giá trị duy nhất ($v = \text{tf.reduce_mean}(S)$). Cuối cùng, một bản đồ đặc trưng bổ sung được thêm vào mỗi trường hợp trong batch và được điền bằng giá trị đã tính ($\text{tf.concat}([\text{inputs}, \text{tf.fill}([\text{batch_size}, \text{height}, \text{width}, 1], v)], \text{axis}=-1)$). Điều này giúp ích như thế nào? Chà, nếu bộ tạo tạo ra hình ảnh ít đa dạng, thì sẽ có độ lệch chuẩn nhỏ trên các bản đồ đặc trưng trong bộ phân biệt. Nhờ lớp này, bộ phân biệt sẽ dễ dàng truy cập vào thống kê này, khiến nó ít bị đánh lừa bởi một bộ tạo tạo ra quá ít đa dạng. Điều này sẽ khuyến khích bộ tạo tạo ra các đầu ra đa dạng hơn, giảm nguy cơ sụp chế độ.
- Tốc độ học bằng nhau (Equalized learning rate):** Khởi tạo tất cả các trọng số bằng cách sử dụng phân phối Gaussian với trung bình 0 và độ lệch chuẩn 1 thay vì sử dụng khởi tạo He. Tuy nhiên, các trọng số được thu nhỏ trong thời gian chạy (tức là mỗi khi

lớp được thực thi) bởi cùng một hệ số như trong khởi tạo He: chúng được chia cho $\sqrt{2 / n_inputs}$, trong đó n_inputs là số đầu vào của lớp. Bài báo đã chứng minh rằng kỹ thuật này cải thiện đáng kể hiệu suất của GAN khi sử dụng RMSProp, Adam hoặc các bộ tối ưu hóa gradient thích ứng khác. Thật vậy, các bộ tối ưu hóa này chuẩn hóa các cập nhật gradient bằng độ lệch chuẩn ước tính của chúng (xem Chương 11), vì vậy các tham số có dải động lớn hơn sẽ mất nhiều thời gian hơn để huấn luyện, trong khi các tham số có dải động nhỏ có thể được cập nhật quá nhanh, dẫn đến mất ổn định. Bằng cách thay đổi tỷ lệ trọng số như một phần của chính mô hình thay vì chỉ thay đổi tỷ lệ chúng khi khởi tạo, cách tiếp cận này đảm bảo rằng dải động là như nhau cho tất cả các tham số trong suốt quá trình huấn luyện, vì vậy tất cả chúng đều học với cùng tốc độ. Điều này vừa tăng tốc vừa ổn định quá trình huấn luyện.

- **Lớp chuẩn hóa theo pixel (Pixelwise normalization layer):** Được thêm vào sau mỗi lớp tích chập trong bộ tạo. Nó chuẩn hóa mỗi kích hoạt dựa trên tất cả các kích hoạt trong cùng một hình ảnh và ở cùng một vị trí, nhưng trên tất cả các kênh (chia cho căn bậc hai của kích hoạt bình phương trung bình). Trong mã TensorFlow, điều này là `inputs / tf.sqrt(tf.reduce_mean(tf.square(X), axis=-1, keepdims=True) + 1e-8)` (thuật ngữ làm mượt $1e-8$ là cần thiết để tránh chia cho 0). Kỹ thuật này tránh sự bùng nổ trong các kích hoạt do sự cạnh tranh quá mức giữa bộ tạo và bộ phân biệt.

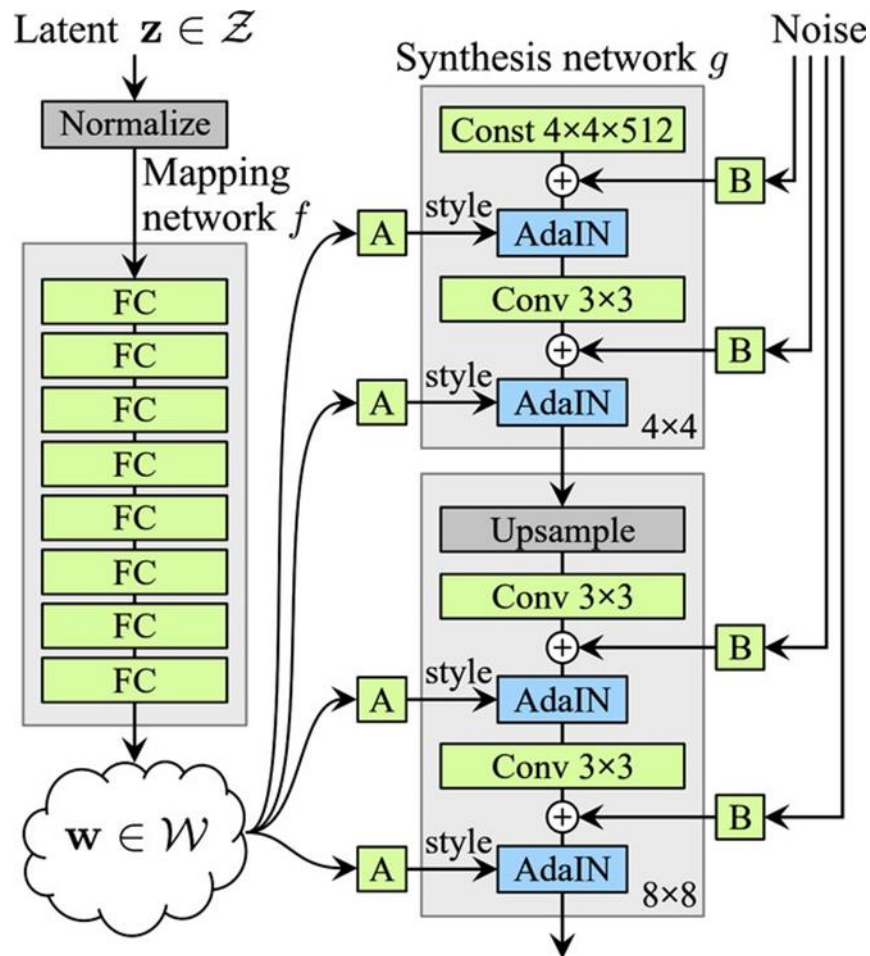
Sự kết hợp của tất cả các kỹ thuật này đã cho phép các tác giả tạo ra những hình ảnh khuôn mặt độ phân giải cao cực kỳ thuyết phục. Nhưng chính xác thì chúng ta gọi “thuyết phục” là gì? Đánh giá là một trong những thách thức lớn khi làm việc với GAN: mặc dù có thể tự động đánh giá sự đa dạng của các hình ảnh được tạo ra, nhưng việc đánh giá chất lượng của chúng là một nhiệm vụ khó khăn và chủ quan hơn nhiều. Một kỹ thuật là sử dụng những người đánh giá, nhưng điều này tốn kém và tốn thời gian. Vì vậy, các tác giả đã đề xuất đo lường sự tương đồng giữa cấu trúc hình ảnh cục bộ của các hình ảnh được tạo ra và các hình ảnh huấn luyện, xem xét mọi tỷ lệ. Ý tưởng này đã dẫn họ đến một đổi mới đột phá khác: StyleGANs.

StyleGANs

Công nghệ tiên tiến trong việc tạo ra hình ảnh độ phân giải cao một lần nữa được đội ngũ Nvidia đó phát triển trong một bài báo năm 2018 đã giới thiệu kiến trúc StyleGAN phổ biến. Các tác giả đã sử dụng các kỹ thuật chuyển đổi phong cách trong bộ tạo để đảm bảo rằng các hình ảnh được tạo ra có cùng cấu trúc cục bộ với hình ảnh huấn luyện, ở mọi tỷ lệ, cải thiện đáng kể chất lượng của hình ảnh được tạo ra. Bộ phân biệt và hàm mất không được sửa đổi, chỉ có bộ tạo. Một bộ tạo StyleGAN bao gồm hai mạng (xem Hình 17-19):

- **Mạng ánh xạ (Mapping network):** Một MLP tám lớp ánh xạ các biểu diễn tiềm ẩn z (tức là các mã hóa) thành một vector w . Vector này sau đó được truyền qua nhiều phép biến đổi affine (tức là các lớp Dense không có hàm kích hoạt, được biểu thị bằng các hộp “A” trong Hình 17-19), tạo ra nhiều vector. Các vector này kiểm soát phong cách của hình ảnh được tạo ở các cấp độ khác nhau, từ kết cấu chi tiết (ví dụ: màu tóc) đến các đặc trưng cấp cao (ví dụ: người lớn hay trẻ em). Tóm lại, mạng ánh xạ ánh xạ các mã hóa thành nhiều vector phong cách.

- Mạng tổng hợp (Synthesis network):** Chịu trách nhiệm tạo ra các hình ảnh. Nó có một đầu vào đã học không đổi (nói rõ hơn, đầu vào này sẽ không đổi sau khi huấn luyện, nhưng trong quá trình huấn luyện nó vẫn được điều chỉnh bởi lan truyền ngược). Nó xử lý đầu vào này thông qua nhiều lớp tích chập và lớp tăng mẫu, như trước đây, nhưng có hai điểm khác biệt. Thứ nhất, một số nhiễu được thêm vào đầu vào và vào tất cả các đầu ra của các lớp tích chập (trước hàm kích hoạt). Thứ hai, mỗi lớp nhiễu được theo sau bởi một lớp chuẩn hóa thể hiện thích ứng (AdaIN): nó chuẩn hóa từng bản đồ đặc trưng một cách độc lập (bằng cách trừ đi giá trị trung bình của bản đồ đặc trưng và chia cho độ lệch chuẩn của nó), sau đó nó sử dụng vector phong cách để xác định tỷ lệ và độ lệch của mỗi bản đồ đặc trưng (vector phong cách chứa một tỷ lệ và một thuật ngữ thiên vị cho mỗi bản đồ đặc trưng).



Hình 17-19. Kiến trúc bộ tạo của StyleGAN (một phần của Hình 1 từ bài báo StyleGAN)

Ý tưởng thêm nhiễu độc lập với các mã hóa là rất quan trọng. Một số phần của hình ảnh khá ngẫu nhiên, chẳng hạn như vị trí chính xác của mỗi nốt ruồi hay sợi tóc. Trong các GAN trước đây, sự ngẫu nhiên này phải đến từ các mã hóa hoặc là một số nhiễu giả ngẫu nhiên do chính bộ tạo tạo ra. Nếu nó đến từ các mã hóa, điều đó có nghĩa là bộ tạo phải dành một phần đáng kể sức mạnh biểu diễn của các mã hóa để lưu trữ nhiễu, điều này khá lãng phí. Hơn nữa, nhiễu phải có khả năng chảy qua mạng và đến các lớp cuối cùng của bộ tạo: điều này dường

như là một ràng buộc không cần thiết có lẽ đã làm chậm quá trình huấn luyện. Và cuối cùng, một số hiện tượng hình ảnh có thể xuất hiện vì cùng một nhiễu được sử dụng ở các cấp độ khác nhau. Nếu thay vào đó, bộ tạo cố gắng tạo ra nhiễu giả ngẫu nhiên của riêng nó, nhiễu này có thể không trông rất thuyết phục, dẫn đến nhiều hiện tượng hình ảnh hơn. Thêm vào đó, một phần trọng số của bộ tạo sẽ được dành để tạo ra nhiễu giả ngẫu nhiên, điều này lại có vẻ lãng phí. Bằng cách thêm các đầu vào nhiễu bổ sung, tất cả các vấn đề này được tránh; GAN có thể sử dụng nhiễu được cung cấp để thêm lượng ngẫu nhiên thích hợp vào mỗi phần của hình ảnh.

Nhiễu được thêm vào khác nhau cho mỗi cấp độ. Mỗi đầu vào nhiễu bao gồm một bản đồ đặc trưng duy nhất đầy nhiễu Gaussian, được lan truyền đến tất cả các bản đồ đặc trưng (ở cấp độ đã cho) và được chia tỷ lệ bằng cách sử dụng các hệ số tỷ lệ đã học trên mỗi đặc trưng (điều này được biểu thị bằng các hộp “B” trong Hình 17-19) trước khi nó được thêm vào.

Cuối cùng, StyleGAN sử dụng một kỹ thuật gọi là **chuẩn hóa trộn (mixing regularization)** (hoặc trộn kiểu - style mixing), trong đó một tỷ lệ phần trăm của các hình ảnh được tạo ra bằng cách sử dụng hai mã hóa khác nhau. Cụ thể, các mã hóa c_1 và c_2 được gửi qua mạng ánh xạ, tạo ra hai vector kiểu w_1 và w_2 . Sau đó, mạng tổng hợp tạo ra một hình ảnh dựa trên các kiểu w_1 cho các cấp độ đầu tiên và các kiểu w_2 cho các cấp độ còn lại. Cấp độ cắt được chọn ngẫu nhiên.

Điều này ngăn mạng giả định rằng các kiểu ở các cấp độ liên kề có tương quan, điều này lần lượt khuyến khích tính cục bộ trong GAN, nghĩa là mỗi vector kiểu chỉ ảnh hưởng đến một số lượng giới hạn các đặc điểm trong hình ảnh được tạo ra.

Có rất nhiều loại GAN khác nhau đến nỗi sẽ cần cả một cuốn sách để bao quát tất cả. Hy vọng phần giới thiệu này đã cung cấp cho bạn những ý tưởng chính, và quan trọng nhất là mong muốn tìm hiểu thêm. Hãy tiếp tục và tự triển khai GAN của riêng bạn, và đừng nản lòng nếu lúc đầu nó gặp khó khăn trong việc học: thật không may, điều này là bình thường, và sẽ cần khá nhiều kiên nhẫn để làm cho nó hoạt động, nhưng kết quả sẽ xứng đáng. Nếu bạn đang gặp khó khăn với một chi tiết triển khai, có rất nhiều triển khai Keras hoặc TensorFlow mà bạn có thể tham khảo. Trên thực tế, nếu tất cả những gì bạn muốn là có được một số kết quả tuyệt vời một cách nhanh chóng, thì bạn chỉ cần sử dụng một mô hình đã được huấn luyện trước (ví dụ: có các mô hình StyleGAN đã được huấn luyện trước có sẵn cho Keras).

Bây giờ chúng ta đã kiểm tra autoencoder và GAN, hãy xem xét một loại kiến trúc cuối cùng: mô hình khuếch tán.

Mô hình khuếch tán (Diffusion Models)

Những ý tưởng đằng sau các mô hình khuếch tán đã tồn tại trong nhiều năm, nhưng chúng lần đầu tiên được chính thức hóa dưới dạng hiện đại trong một bài báo năm 2015 của Jascha Sohl-Dickstein và cộng sự từ Đại học Stanford và UC Berkeley. Các tác giả đã áp dụng các công cụ từ nhiệt động lực học để mô hình hóa một quá trình khuếch tán, tương tự như một giọt sữa khuếch tán trong một cốc trà. Ý tưởng cốt lõi là huấn luyện một mô hình để học quá trình ngược lại: bắt đầu từ trạng thái hoàn toàn hỗn hợp, và dần dần “tách” sữa ra khỏi trà.

Sử dụng ý tưởng này, họ đã đạt được kết quả đầy hứa hẹn trong việc tạo hình ảnh, nhưng vì GANs tạo ra hình ảnh thuyết phục hơn vào thời điểm đó, các mô hình khuếch tán đã không nhận được nhiều sự chú ý.

Sau đó, vào năm 2020, Jonathan Ho và cộng sự, cũng từ UC Berkeley, đã xây dựng được một mô hình khuếch tán có khả năng tạo ra hình ảnh có độ chân thực cao, mà họ gọi là mô hình xác suất khuếch tán khử nhiễu (denoising diffusion probabilistic model - DDPM). Vài tháng sau, một bài báo năm 2021 của các nhà nghiên cứu OpenAI Alex Nichol và Prafulla Dhariwal đã phân tích kiến trúc DDPM và đề xuất một số cải tiến cho phép DDPMs cuối cùng đánh bại GANs: DDPMs không chỉ dễ huấn luyện hơn nhiều so với GANs, mà các hình ảnh được tạo ra còn đa dạng hơn và có chất lượng cao hơn nữa. Nhược điểm chính của DDPMs, như bạn sẽ thấy, là chúng mất rất nhiều thời gian để tạo ra hình ảnh, so với GANs hoặc VAEs.

Vậy chính xác thì DDPM hoạt động như thế nào? Chà, giả sử bạn bắt đầu với một bức ảnh con mèo (như cái bạn sẽ thấy trong Hình 17-20), ký hiệu x_0 , và ở mỗi bước thời gian t bạn thêm một chút nhiễu Gaussian vào hình ảnh, với trung bình 0 và phương sai β_t . Nhiễu này độc lập cho mỗi pixel: chúng ta gọi nó là đẳng hướng. Bạn đầu tiên có được hình ảnh x_1 , sau đó x_2 , v.v., cho đến khi con mèo hoàn toàn bị nhiễu che khuất, không thể nhìn thấy. Bước thời gian cuối cùng được ký hiệu là T . Trong bài báo DDPM gốc, các tác giả đã sử dụng $T = 1.000$, và họ đã lên lịch phương sai β_t theo cách mà tín hiệu mèo mờ dần tuyến tính giữa các bước thời gian 0 và T . Trong bài báo DDPM cải tiến, T được nâng lên 4.000, và lịch phương sai được điều chỉnh để thay đổi chậm hơn ở đầu và cuối. Tóm lại, chúng ta đang dần dần nhấn chìm con mèo trong nhiễu: đây được gọi là **quá trình tiến (forward process)**.

Khi chúng ta thêm ngày càng nhiều nhiễu Gaussian vào quá trình tiến, phân phối của các giá trị pixel ngày càng trở nên Gaussian hơn. Một chi tiết quan trọng tôi đã bỏ qua là các giá trị pixel được thay đổi tỷ lệ nhẹ ở mỗi bước, bằng một yếu tố $\sqrt{1 - \beta_t}$. Điều này đảm bảo rằng trung bình của các giá trị pixel dần dần tiến đến 0, vì yếu tố tỷ lệ nhỏ hơn 1 một chút (hãy tưởng tượng liên tục nhân một số với 0.99). Nó cũng đảm bảo rằng phương sai sẽ dần dần hội tụ về 1. Điều này là do độ lệch chuẩn của các giá trị pixel cũng được chia tỷ lệ bởi $\sqrt{1 - \beta_t}$, vì vậy phương sai được chia tỷ lệ bởi $1 - \beta_t$ (tức là bình phương của yếu tố tỷ lệ). Nhưng phương sai không thể giảm xuống 0 vì chúng ta đang thêm nhiễu Gaussian với phương sai β_t ở mỗi bước. Và vì phương sai cộng lại khi bạn tổng hợp các phân phối Gaussian, bạn có thể thấy rằng phương sai chỉ có thể hội tụ về $1 - \beta_t + \beta_t = 1$.

Quá trình khuếch tán tiến được tóm tắt trong Công thức 17-5. Phương trình này sẽ không dạy bạn bất cứ điều gì mới về quá trình tiến, nhưng nó hữu ích để hiểu loại ký hiệu toán học này, vì nó thường được sử dụng trong các bài báo ML. Phương trình này định nghĩa phân phối xác suất q của x_t cho trước x_{t-1} là một phân phối Gaussian với trung bình x_{t-1} nhân với yếu tố tỷ lệ, và với ma trận hiệp phương sai bằng $\beta_t \mathbf{I}$. Đây là ma trận đơn vị $\mathbf{I}$ nhân với β_t , có nghĩa là nhiễu là đẳng hướng với phương sai β_t .

Công thức 17-5. Phân phối xác suất q của quá trình khuếch tán tiến

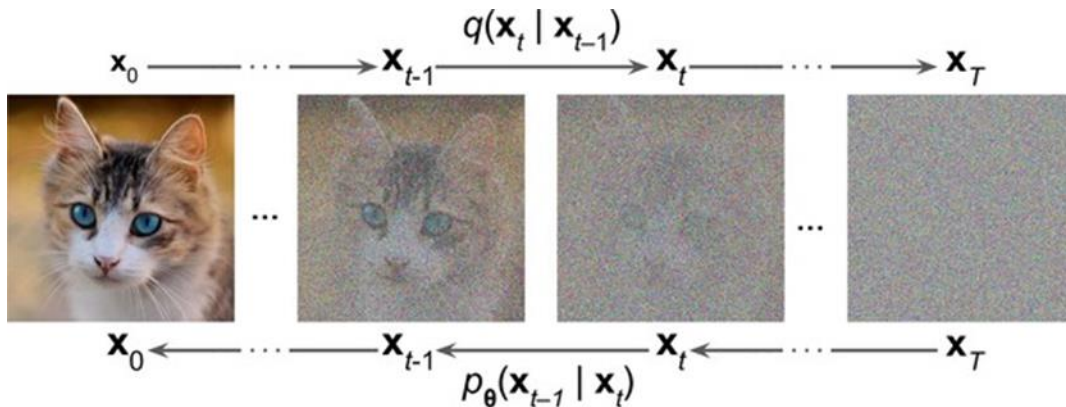
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

Điều thú vị là, có một lối tắt cho quá trình tiến: có thể lấy mẫu một hình ảnh x_t cho trước x_0 mà không cần phải tính $x_1, x_2, \dots, x_{t-1}$ trước. Thật vậy, vì tổng của nhiều phân phối Gaussian cũng là một phân phối Gaussian, tất cả nhiễu có thể được thêm vào chỉ trong một lần bằng cách sử dụng Công thức 17-6. Đây là phương trình chúng ta sẽ sử dụng, vì nó nhanh hơn nhiều.

Công thức 17-6. Lối tắt cho quá trình khuếch tán tiến

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

Mục tiêu của chúng ta, tất nhiên, không phải là nhấn chìm mèo trong nhiễu. Ngược lại, chúng ta muốn tạo ra nhiều con mèo mới! Chúng ta có thể làm như vậy bằng cách huấn luyện một mô hình có thể thực hiện quá trình ngược lại: đi từ x_t đến x_{t-1} . Sau đó, chúng ta có thể sử dụng nó để loại bỏ một chút nhiễu khỏi một hình ảnh, và lặp lại thao tác nhiều lần cho đến khi tất cả nhiễu biến mất. Nếu chúng ta huấn luyện mô hình trên một tập dữ liệu chứa nhiều hình ảnh mèo, thì chúng ta có thể cung cấp cho nó một bức ảnh hoàn toàn đầy nhiễu Gaussian, và mô hình sẽ dần dần tạo ra một con mèo hoàn toàn mới (xem Hình 17-20).



Hình 17-20. Quá trình tiến q và quá trình ngược p

Okay, bây giờ chúng ta sẽ bắt đầu viết code! Việc đầu tiên cần làm là lập trình quá trình chuyển tiếp (forward process). Để làm điều này, chúng ta cần triển khai lịch trình phương sai. Làm thế nào chúng ta có thể kiểm soát tốc độ con mèo biến mất? Ban đầu, 100% phương sai đến từ hình ảnh con mèo gốc. Sau đó, tại mỗi bước thời gian t , phương sai được nhân với $1 - \beta_t$ và nhiễu được thêm vào. Vì vậy, phần phương sai đến từ phân phối ban đầu co lại bằng một hệ số $1 - \beta_t$ ở mỗi bước. Nếu chúng ta định nghĩa $a_t = 1 - \beta_t$, thì sau t bước thời gian, hình ảnh con mèo sẽ được nhân với một hệ số $\bar{\alpha}_t = a_1 a_2 \dots a_t = \prod_{i=1}^t a_i$. Đây là “hệ số tín hiệu con mèo” $\bar{\alpha}_t$ mà chúng ta muốn lên lịch trình để nó giảm dần từ 1 xuống 0 một cách từ từ giữa bước thời gian 0 và T .

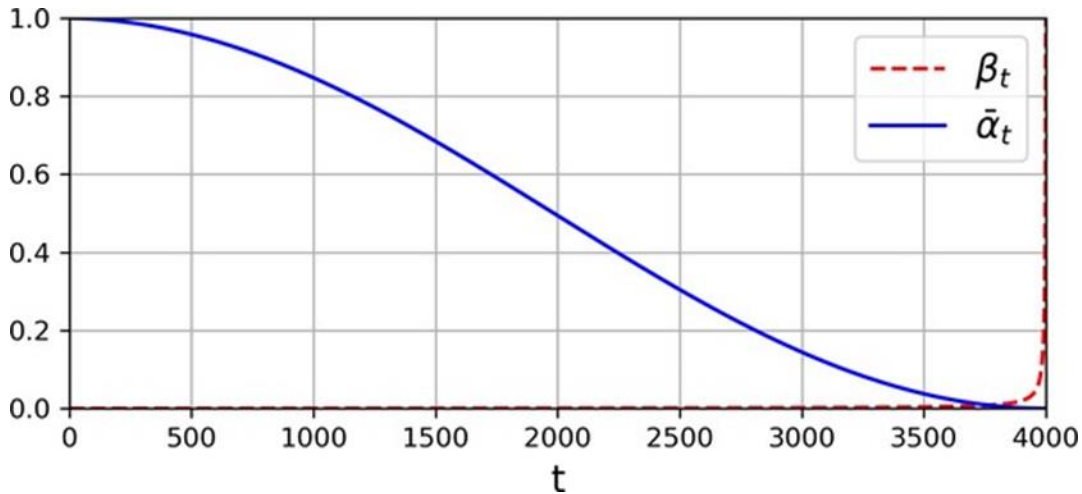
Trong bài báo DDPM được cải tiến, các tác giả đã lên lịch trình cho a_t theo **Công thức 17-7**. Lịch trình này được thể hiện trong Hình 17-21.

Công thức 17-7: Phương trình lịch trình phương sai cho quá trình khuếch tán chuyển tiếp

$$\beta_t = 1 - \frac{\alpha_t^-}{\alpha_{t-1}^-}, \text{ với } \alpha_t^- = \frac{f(t)}{f(0)} \text{ và } f(t) = \left(\cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right) \right)^2$$

Trong các phương trình này:

- s là một giá trị nhỏ ngăn β_t không quá nhỏ gần $t = 0$. Trong bài báo, các tác giả đã sử dụng $s = 0.008$.
- β_t được cắt để không lớn hơn 0.999, nhằm tránh sự mất ổn định gần $t = T$.



Hình 17-21. Lịch phương sai nhiễu β_t , và phương sai tín hiệu còn lại α_{bar_t}

Hãy tạo một hàm nhỏ để tính α_t , β_t , và α_{bar_t} , và gọi nó với $T = 4.000$:

```
import numpy as np
import tensorflow as tf

def variance_schedule(T, s=0.008, max_beta=0.999):
    t = np.arange(T + 1)
    f = np.cos((t / T + s) / (1 + s) * np.pi / 2) ** 2
    alpha = np.clip(f[1:] / f[:-1], 1 - max_beta, 1)
    alpha = np.append(1, alpha).astype(np.float32) # add  $\alpha_0 = 1$ 
    beta = 1 - alpha
    alpha_cumprod = np.cumprod(alpha)

    return alpha, alpha_cumprod, beta #  $\alpha_t$ ,  $\alpha_t^-$ ,  $\beta_t$  for  $t = 0$  to  $T$ 

T = 4000
alpha, alpha_cumprod, beta = variance_schedule(T)
```

Để huấn luyện mô hình của chúng ta đảo ngược quá trình khuếch tán, chúng ta sẽ cần hình ảnh bị nhiễu từ các bước thời gian khác nhau của quá trình tiến. Để làm điều này, hãy tạo một hàm `prepare_batch()` sẽ lấy một batch hình ảnh sạch từ tập dữ liệu và chuẩn bị chúng:

```

import tensorflow as tf
import numpy as np

def prepare_batch(X):
    # Đảm bảo X là float32 và mở rộng từ [0, 1] sang [-1, 1]
    X = tf.cast(X[..., tf.newaxis], tf.float32) * 2 - 1
    X_shape = tf.shape(X)
    # Lấy một bước thời gian ngẫu nhiên cho mỗi hình ảnh trong batch
    t = tf.random.uniform([X_shape[0]], minval=1, maxval=T + 1, dtype=tf.int32)

    # Lấy alpha_cumprod tương ứng với bước thời gian t
    alpha_cm = tf.gather(alpha_cumprod, t)
    # Định hình lại alpha_cm để có thể broadcast với X (ví dụ: [batch_size, 1, 1, 1])
    alpha_cm = tf.reshape(alpha_cm, [X_shape[0]] + [1] * (len(X_shape) - 1))
    noise = tf.random.normal(X_shape)

    # Áp dụng quá trình khuếch tán tiến (Công thức 17-6)
    X_noisy = alpha_cm ** 0.5 * X + (1 - alpha_cm) ** 0.5 * noise

    return {
        "X_noisy": X_noisy,
        "time": t,
    }, noise

```

Hãy xem qua đoạn mã này:

- Để đơn giản, chúng ta sẽ sử dụng Fashion MNIST, vì vậy hàm phải thêm một trục kênh trước. Nó cũng sẽ giúp thay đổi tỷ lệ các giá trị pixel từ -1 đến 1, để nó gần hơn với phân phối Gaussian cuối cùng với trung bình 0 và phương sai 1.
- Tiếp theo, hàm tạo t, một vector chứa một bước thời gian ngẫu nhiên cho mỗi hình ảnh trong batch, từ 1 đến T.
- Sau đó, nó sử dụng `tf.gather()` để lấy giá trị của `alpha_cumprod` cho mỗi bước thời gian trong vector t. Điều này mang lại cho chúng ta vector `alpha_cm`, chứa một giá trị α_{bar_t} cho mỗi hình ảnh.
- Dòng tiếp theo định hình lại `alpha_cm` từ [kích thước batch] thành [kích thước batch, 1, 1, 1]. Điều này là cần thiết để đảm bảo `alpha_cm` có thể được broadcast với batch X.
- Sau đó chúng ta tạo ra một số nhiễu Gaussian với trung bình 0 và phương sai 1.
- Cuối cùng, chúng ta sử dụng Công thức 17-6 để áp dụng quá trình khuếch tán vào hình ảnh. Lưu ý rằng `x ** 0.5` bằng căn bậc hai của x. Hàm trả về một tuple chứa đầu vào và mục tiêu. Các đầu vào được biểu diễn dưới dạng một dict Python chứa các hình ảnh bị nhiễu và các bước thời gian được sử dụng để tạo ra chúng. Các mục tiêu là nhiễu Gaussian được sử dụng để tạo ra mỗi hình ảnh.

Tiếp theo, chúng ta sẽ tạo một tập dữ liệu huấn luyện và một tập xác thực sẽ áp dụng hàm `prepare_batch()` cho mọi batch. Như trước đó, `X_train` và `X_valid` chứa các hình ảnh Fashion MNIST với các giá trị pixel từ 0 đến 1:

```
import tensorflow as tf

def prepare_dataset(X, batch_size=32, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices(X)

    if shuffle:
        ds = ds.shuffle(buffer_size=10_000)

    return ds.batch(batch_size).map(prepare_batch).prefetch(1)

# Giả định X_train và X_valid đã được định nghĩa
# train_set = prepare_dataset(X_train, batch_size=32, shuffle=True)
# valid_set = prepare_dataset(X_valid, batch_size=32)
```

Bây giờ chúng ta đã sẵn sàng để xây dựng mô hình khuếch tán thực tế. Nó có thể là bất kỳ mô hình nào bạn muốn, miễn là nó nhận các hình ảnh bị nhiễu và các bước thời gian làm đầu vào, và dự đoán nhiễu cần trừ khỏi hình ảnh đầu vào:

```
import tensorflow as tf

def build_diffusion_model():
    X_noisy = tf.keras.layers.Input(shape=[28, 28, 1], name="X_noisy")
    time_input = tf.keras.layers.Input(shape=[], dtype=tf.int32, name="time")

    # [...] # xây dựng mô hình dựa trên hình ảnh bị nhiễu và các bước thời gian
    # outputs = [...] # dự đoán nhiễu (cùng hình dạng với hình ảnh đầu vào)
    # return tf.keras.Model(inputs=[X_noisy, time_input], outputs=[outputs])

    # Đây là một ví dụ đơn giản để minh họa cấu trúc, không phải kiến trúc U-Net thực tế
    # Bạn sẽ thay thế phần này bằng kiến trúc U-Net được mô tả
    x = tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu")(X_noisy)
    x = tf.keras.layers.MaxPool2D(pool_size=2)(x)
    x = tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    # Để đơn giản, bỏ qua việc tích hợp time_input vào đây
    # Trong một DDPM thực tế, time_input sẽ được mã hóa và nhúng vào mạng

    # Giả định đầu ra có cùng hình dạng với đầu vào nhiễu
    outputs = tf.keras.layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)
    outputs = tf.keras.layers.Conv2DTranspose(1, 3, padding="same", activation="linear")(outputs) # Đầu ra là nhiễu, không phải ảnh, nên dùng linear
```

```

return tf.keras.Model(inputs=[X_noisy, time_input], outputs=[outputs])

# model = build_diffusion_model()

```

Các tác giả của DDPM đã sử dụng kiến trúc U-Net đã được sửa đổi, có nhiều điểm tương đồng với kiến trúc FCN mà chúng ta đã thảo luận trong Chương 14 về phân đoạn ngữ nghĩa: đó là một mạng thần kinh tích chập dần dần giảm mẫu hình ảnh đầu vào, sau đó dần dần tăng mẫu chúng trở lại, với các kết nối bỏ qua (skip connections) từ mỗi cấp độ của phần giảm mẫu sang cấp độ tương ứng trong phần tăng mẫu. Để tính đến các bước thời gian, họ đã mã hóa chúng bằng cách sử dụng cùng kỹ thuật như mã hóa vị trí trong kiến trúc transformer (xem Chương 16). Ở mỗi cấp độ trong kiến trúc U-Net, họ truyền các mã hóa thời gian này qua các lớp Dense và cấp chúng cho U-Net. Cuối cùng, họ cũng sử dụng các lớp chú ý đa đầu (multi-head attention) ở các cấp độ khác nhau. Xem sổ tay của chương này để biết một triển khai cơ bản, hoặc <https://hml.info/ddpmcode> để biết triển khai chính thức: nó dựa trên TF 1.x, đã lỗi thời, nhưng khá dễ đọc.

Chúng ta bây giờ có thể huấn luyện mô hình bình thường. Các tác giả lưu ý rằng việc sử dụng hàm mất MAE hoạt động tốt hơn MSE. Bạn cũng có thể sử dụng hàm mất Huber:

```

import tensorflow as tf

# Giả định model đã được xây dựng bằng build_diffusion_model()
# Giả định train_set và valid_set đã được chuẩn bị bằng prepare_dataset()

# model.compile(loss=tf.keras.losses.Huber(), optimizer="nadam")
# history = model.fit(train_set, validation_data=valid_set, epochs=100)

```

Sau khi mô hình được huấn luyện, bạn có thể sử dụng nó để tạo hình ảnh mới. Thật không may, không có lỗi tắt trong quá trình khuếch tán ngược, vì vậy bạn phải lấy mẫu x_T ngẫu nhiên từ một phân phối Gaussian với trung bình 0 và phương sai 1, sau đó truyền nó cho mô hình để dự đoán nhiễu; trừ nó khỏi hình ảnh bằng cách sử dụng Công thức 17-8, và bạn sẽ có x_{T-1} . Lặp lại quá trình thêm 3.999 lần nữa cho đến khi bạn có x_0 : nếu mọi thứ diễn ra tốt đẹp, nó sẽ trông giống như một hình ảnh Fashion MNIST thông thường!

Công thức 17-8. Đi ngược một bước trong quá trình khuếch tán

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sqrt{\beta_t} \mathbf{z}$$

Trong phương trình này, $\epsilon_{\theta}(\mathbf{x}_t, t)$ biểu thị nhiễu được dự đoán bởi mô hình cho trước hình ảnh đầu vào $\mathbf{x}_t$ và bước thời gian t . θ biểu thị các tham số mô hình. Hơn nữa, $\mathbf{z}$ là nhiễu Gaussian với trung bình 0 và phương sai 1. Điều này làm cho quá trình ngược trở nên ngẫu nhiên: nếu bạn chạy nó nhiều lần, bạn sẽ nhận được các hình ảnh khác nhau.

Hãy viết một hàm triển khai quá trình ngược này, và gọi nó để tạo một vài hình ảnh:

```

import tensorflow as tf
import numpy as np

```

```

def generate(model, batch_size=32):
    # Lấy mẫu xT ngẫu nhiên từ phân phối Gaussian với trung bình 0 và phương
    sai 1
    X = tf.random.normal([batch_size, 28, 28, 1])

    for t in range(T, 0, -1): # Lặp từ T xuống 1
        # Nhiễu z. Đối với bước cuối cùng (t=1), z là một tensor zero
        noise = (tf.random.normal if t > 1 else tf.zeros)(tf.shape(X))

        # Dự đoán nhiễu (epsilon_theta)
        X_noisy = model({"X_noisy": X, "time": tf.constant([t] * batch_size)})
    )

    # Áp dụng Công thức 17-8
    # Các biến alpha, beta, alpha_cumprod phải có sẵn từ biến toàn cục
    X = (
        1 / alpha[t] ** 0.5
        * (X - beta[t] / (1 - alpha_cumprod[t]) ** 0.5 * X_noisy)
        + (1 - alpha[t]) ** 0.5 * noise
    )

    return X

# X_gen = generate(model) # Các hình ảnh được tạo

```

Điều này có thể mất một hoặc hai phút. Đó là nhược điểm chính của các mô hình khuếch tán: tạo hình ảnh chậm vì mô hình cần được gọi nhiều lần. Có thể làm điều này nhanh hơn bằng cách sử dụng giá trị T nhỏ hơn, hoặc bằng cách sử dụng cùng một dự đoán mô hình cho một vài bước cùng một lúc, nhưng các hình ảnh kết quả có thể không đẹp bằng. Điều đó nói rằng, bất chấp hạn chế về tốc độ này, các mô hình khuếch tán vẫn tạo ra hình ảnh chất lượng cao và đa dạng, như bạn có thể thấy trong Hình 17-22.



Hình 17-22. Hình ảnh được tạo bởi DDPM

Các mô hình khuếch tán đã có những tiến bộ vượt bậc gần đây. Đặc biệt, một bài báo được xuất bản vào tháng 12 năm 2021 bởi Robin Rombach, Andreas Blattmann, và cộng sự, đã giới thiệu **mô hình khuếch tán tiềm ẩn (latent diffusion models)**, trong đó quá trình khuếch tán diễn ra trong không gian tiềm ẩn, thay vì không gian pixel. Để đạt được điều này, một autoencoder mạnh mẽ được sử dụng để nén mỗi hình ảnh huấn luyện thành một không gian tiềm ẩn nhỏ hơn nhiều, nơi quá trình khuếch tán diễn ra, sau đó autoencoder được sử dụng để giải nén biểu diễn tiềm ẩn cuối cùng, tạo ra hình ảnh đầu ra. Điều này làm tăng tốc đáng kể việc tạo hình ảnh, và giảm thời gian và chi phí huấn luyện một cách đáng kể. Quan trọng là, chất lượng của các hình ảnh được tạo ra là xuất sắc.

Hơn nữa, các nhà nghiên cứu cũng đã điều chỉnh các kỹ thuật điều kiện hóa khác nhau để hướng dẫn quá trình khuếch tán bằng cách sử dụng các lời nhắc văn bản, hình ảnh hoặc bất kỳ đầu vào nào khác. Điều này giúp có thể nhanh chóng tạo ra một hình ảnh độ phân giải cao, đẹp mắt về một con kỳ nhông đang đọc sách, hoặc bất cứ thứ gì khác bạn có thể thích. Bạn cũng có thể điều kiện hóa quá trình tạo hình ảnh bằng cách sử dụng một hình ảnh đầu vào. Điều này cho phép nhiều ứng dụng, chẳng hạn như vẽ ngoài (outpainting) — trong đó một hình ảnh đầu vào được mở rộng ra ngoài biên giới của nó — hoặc vẽ trong (inpainting) — trong đó các lỗ trên một hình ảnh được lấp đầy.

Cuối cùng, một mô hình khuếch tán tiềm ẩn mạnh mẽ đã được huấn luyện trước có tên Stable Diffusion đã được mã nguồn mở vào tháng 8 năm 2022 bởi sự hợp tác giữa LMU Munich và một vài công ty, bao gồm StabilityAI và Runway, với sự hỗ trợ từ EleutherAI và LAION. Vào tháng 9 năm 2022, nó đã được chuyển sang TensorFlow và được bao gồm trong KerasCV, một thư viện thị giác máy tính được xây dựng bởi nhóm Keras. Bây giờ bất kỳ ai cũng có thể tạo ra những hình ảnh tuyệt vời trong vài giây, miễn phí, ngay cả trên một máy tính xách tay thông thường (xem bài tập cuối cùng trong chương này). Khả năng là vô tận!

Trong chương tiếp theo, chúng ta sẽ chuyển sang một nhánh hoàn toàn khác của học sâu: học tăng cường sâu (deep reinforcement learning).

Bài tập

- Các nhiệm vụ chính mà autoencoder được sử dụng là gì?
- Giả sử bạn muốn huấn luyện một bộ phân loại, và bạn có rất nhiều dữ liệu huấn luyện không được gán nhãn nhưng chỉ có vài nghìn trường hợp được gán nhãn. Autoencoder có thể giúp ích như thế nào? Bạn sẽ tiến hành như thế nào?
- Nếu một autoencoder tái tạo hoàn hảo đầu vào, nó có nhất thiết là một autoencoder tốt không? Làm thế nào bạn có thể đánh giá hiệu suất của một autoencoder?
- Autoencoder thiếu hoàn chỉnh (undercomplete) và thừa hoàn chỉnh (overcomplete) là gì? Rủi ro chính của một autoencoder thiếu hoàn chỉnh quá mức là gì? Còn rủi ro chính của một autoencoder thừa hoàn chỉnh thì sao?
- Bạn làm thế nào để ràng buộc trọng số trong một autoencoder xếp chồng? Mục đích của việc làm như vậy là gì?

- Mô hình sinh là gì? Bạn có thể kể tên một loại autoencoder sinh không?
- GAN là gì? Bạn có thể kể tên một vài nhiệm vụ mà GAN có thể phát huy tác dụng không?
- Những khó khăn chính khi huấn luyện GAN là gì?
- Mô hình khuếch tán giải điều gì? Hạn chế chính của chúng là gì?
- Hãy thử sử dụng autoencoder khử nhiễu để tiền huấn luyện một bộ phân loại hình ảnh. Bạn có thể sử dụng MNIST (tùy chọn đơn giản nhất), hoặc một tập dữ liệu hình ảnh phức tạp hơn như CIFAR10 nếu bạn muốn thử thách lớn hơn. Bất kể tập dữ liệu bạn đang sử dụng là gì, hãy làm theo các bước sau: a. Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra. Huấn luyện một autoencoder khử nhiễu sâu trên toàn bộ tập huấn luyện. b. Kiểm tra xem các hình ảnh có được tái tạo khá tốt không. Trực quan hóa các hình ảnh kích hoạt mạnh nhất mỗi neuron trong lớp mã hóa. c. Xây dựng một DNN phân loại, sử dụng lại các lớp thấp hơn của autoencoder. Huấn luyện nó chỉ với 500 hình ảnh từ tập huấn luyện. Nó hoạt động tốt hơn khi có hoặc không có tiền huấn luyện?
- Huấn luyện một autoencoder biến phân trên tập dữ liệu hình ảnh bạn chọn, và sử dụng nó để tạo hình ảnh. Ngoài ra, bạn có thể thử tìm một tập dữ liệu không được gán nhãn mà bạn quan tâm và xem liệu bạn có thể tạo ra các mẫu mới không.
- Huấn luyện một DCGAN để giải quyết tập dữ liệu hình ảnh bạn chọn, và sử dụng nó để tạo hình ảnh. Thêm replay kinh nghiệm và xem liệu điều này có giúp ích không. Biến nó thành một GAN có điều kiện nơi bạn có thể kiểm soát lớp được tạo.
- Hãy xem hướng dẫn Stable Diffusion tuyệt vời của KerasCV, và tạo một bản vẽ đẹp về một con kỳ nhông đang đọc sách. Nếu bạn đăng bản vẽ đẹp nhất của mình lên Twitter, vui lòng gắn thẻ tôi tại @aureliengeron. Tôi rất muốn xem các tác phẩm của bạn!

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.

¹ William G. Chase và Herbert A. Simon, “Perception in Chess”, *Cognitive Psychology* 4, số 1 (1973): 55–81. ² Yoshua Bengio và cộng sự, “Greedy Layer-Wise Training of Deep Networks”, *Proceedings of the 19th International Conference on Neural Information Processing Systems* (2006): 153–160. ³ Jonathan Masci và cộng sự, “Stacked Convolutional Auto-Encoders for Hierarchical Feature Extraction”, *Proceedings of the 21st International Conference on Artificial Neural Networks* 1 (2011): 52–59. ⁴ Pascal Vincent và cộng sự, “Extracting and Composing Robust Features with Denoising Autoencoders”, *Proceedings of the 25th International Conference on Machine Learning* (2008): 1096–1103. ⁵ Pascal Vincent và cộng sự, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion”, *Journal of Machine Learning Research* 11 (2010): 3371–3408. ⁶ Diederik Kingma và Max Welling, “Auto-Encoding Variational Bayes”, *arXiv preprint arXiv:1312.6114* (2013). ⁷ Autoencoder biến phân thực tế tổng quát hơn; các mã hóa không giới hạn ở phân phối Gaussian. ⁸ Để biết thêm chi tiết toán học, hãy xem bài báo gốc về autoencoder biến phân, hoặc hướng dẫn tuyệt vời của Carl Doersch (2016). ⁹ Ian

Goodfellow và cộng sự, “Generative Adversarial Nets”, *Proceedings of the 27th International Conference on Neural Information Processing Systems 2* (2014): 2672–2680.¹⁰ Để so sánh các hàm mất GAN chính, hãy xem dự án GitHub tuyệt vời này của Hwalsuk Lee.¹¹ Mario Lucic và cộng sự, “Are GANs Created Equal? A Large-Scale Study”, *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (2018): 698–707.¹² Alec Radford và cộng sự, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”, *arXiv preprint* arXiv:1511.06434 (2015).¹³ Được tái bản với sự cho phép của các tác giả.¹⁴ Mehdi Mirza và Simon Osindero, “Conditional Generative Adversarial Nets”, *arXiv preprint* arXiv:1411.1784 (2014).¹⁵ Tero Karras và cộng sự, “Progressive Growing of GANs for Improved Quality, Stability, and Variation”, *Proceedings of the International Conference on Learning Representations* (2018).¹⁶ Dải động của một biến là tỷ lệ giữa giá trị cao nhất và thấp nhất mà nó có thể nhận.¹⁷ Tero Karras và cộng sự, “A Style-Based Generator Architecture for Generative Adversarial Networks”, *arXiv preprint* arXiv:1812.04948 (2018).¹⁸ Được tái bản với sự cho phép của các tác giả.¹⁹ Jascha Sohl-Dickstein và cộng sự, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”, *arXiv preprint* arXiv:1503.03585 (2015).²⁰ Jonathan Ho và cộng sự, “Denoising Diffusion Probabilistic Models” (2020).²¹ Alex Nichol và Prafulla Dhariwal, “Improved Denoising Diffusion Probabilistic Models” (2021).²² Olaf Ronneberger và cộng sự, “U-Net: Convolutional Networks for Biomedical Image Segmentation”, *arXiv preprint* arXiv:1505.04597 (2015).²³ Robin Rombach, Andreas Blattmann, và cộng sự, “High-Resolution Image Synthesis with Latent Diffusion Models”, *arXiv preprint* arXiv:2112.10752 (2021).